Controlling and Automating a Model Railway with MersAG_Universal, OpenLCB and JMRI

This User Manual is presented in 4 parts each of which is available as a separate PDF download.

Important Note *:*
If you're reading this document directly from the Github repository you may find formatting errors. Please open the PDF in the MersAG_Universal PDF Documentation Folder.

Part 1 gives a summary of the technologies used and why, as model railway enthusiasts, we've chosen to use them. More importantly, it sets out why it gives the railway modelling community a, low cost path to achieving high levels of reliable automation covering "realistic" turnout control and frog switching, signalling, sensing, block detection, lighting, animations and more.

Part 2 is aimed at modellers with little knowledge of electronics and/or, maybe, the fear of soldering!! I'll walk you through making a very simple, cheap "breadboard" to simulate a red/green signal from a simple push button switch. You'll then learn how easy it is, with MersAG_Universal, to integrate this with JMRI. Finally, and with just 2 more components and very little effort, you can add slow-motion servo motor control. Our aim is to let you decide, with very little time and cost, if the MersAG_Universal way looks right for you.

In **Part 3** I'll describe and illustrate the steps I took to build a "node" that can control multiple turnouts, signals etc and detect multiple sensors to control and automate part of my layout model railway layout.

Finally, in **Part 4**, I'll add just 1 more component to enable several nodes to communicate over a CAN bus network that will enable you to control even very large complex layouts reliably.

This User Guide is supported by a series of YouTube Videos at John's : Sentence on your vids and Links to relevant series and Book?.

Part 3 : Building and Configuring at Node

3.1 Introduction 1

3.2 Deciding What Components I'd Need on my "Colliery" Node 2

3.3 Defining Unique Addresses for the Expansion Boards 3

3.4 Positioning Components on the Node Board 4

3.5 Wiring the I2C Bus 5

3.6 Download the MersAG_Universal Package : including I2C Scanner 6

3.7 Test I2C Bus with i2cScanner 7

3.8 Connecting Servos to the PCA9685 Board 8

3.9 Configure MersAG_Universal and Install it on the ESP32 9

3.10 Configure JMRI to "talk" to the ESP32 10

3.11 Configure the Node in PanelPro 11

3.12 Configure and Test the Servo Modules in PanelPro 14

3.13 Planning Pin Connections 16
3.14 Connect the 8 and 6 Channel Relay Boards to the Node Board 16
3.15 Configuring the MCP Boards in PanelPro 17
3.16 Configure and Test Relay Boards to Control Frog Switches 18
3.17 Teaching Servos to Control Frog Polarity through the Relay Switches 19
3.18 Configuring the Red and Green Signal LEDs 22
3.19 Teaching Servos how to Control Signal LEDs 23
3.20 Configure Relay Board to Switch Station and Other Lighting 25
3.21 Configure and Test IR Sensors 26
Appendices 27
Appendix 1 : Layout Diagram 27
Appendix 2 : Colliery Detailed Layout Showing Servo and Signal Descriptions 28
Appendix 3 : Colliery Detailed Layout Showing Sensor Descriptions 29
Appendix 4 : Expansion Boards needed to Control Layout Regions 30
Appendix 5 : Pin Connections Plan 31
Appendix 5 : Pin Connections Plan (Continued) 32
Appendix 6 : Channel Types 33
Appendix 7 : Input / Output Delay Settings 35
References 37

3.1 Introduction

If you're a beginner and haven't yet read Part 1 you might want to take a look as you'll get an overview of many of the terms and technologies I'm using and why. You might want to look through Part 2 as well and maybe have a go at the breadboard exercise if you've not already done so.

In this part I'm going to significantly increase the capabilities of my ESP32 and configure MersAG_Universal to control most of the functions needed to control and automate a model railway. I'll start by building, configuring and testing a node which I'll then install on my model railway and demonstrate control and automation in a real-life example.

This layout currently uses a single Arduino controller to control 26 servo-actuated turnouts/points with relay-actuated frogs through JMRI/PanelPro and a "homegrown" sketch. I realised that, to add signals, sensors, lights, animations etc, would take me way beyond the Arduino's capabilities and stretch my sketch writing ability; probably beyond-breaking point! I was also concerned that, on an 00 gauge, 2.45 x 2.85m layout, wiring distances and complexity would likely lead to interference and unreliability.

Appendix 1 shows a Plan of the current layout and an enlarged detailed plan of the first region that I'm going to upgrade is shown in Appendix 2.

I recognised that, with a few MersAg_Universal nodes, I should be able to achieve slow, more realistic movement of my turnouts and add numerous other features to enable, through JMRI, the significant degree of control and automation I was hoping for.

I'm now going to build a node with my ESP32, expansion and other boards and connect it to a test-bed with servos, frog-controlled relays, signals, sensors and lights. I'll configure my MersAG_Universal sketch with JMRI and set up a layout panel in PanelPro to show the various functions working before installing the node on the Colliery region of my layout.

So before getting started, I needed to clearly define what I needed my node(s) to control and automate on my layout and make sure that I'd have the capacity (number of inputs and outputs) needed.

3.2 Deciding What Components I'd Need on my "Colliery" Node

MersAG_Universal can support :

1 PCA9685 Expansion Board with 1 - 4 MCP23017 Expansion Boards; **or**

2 PCA9685 Expansion Board with 1 - 3 MCP23017 Expansion Boards

Referring to my layout plan I created a spreadsheet to calculate how many PCA9685 and MCP23017 expansion boards I'd need : this is shown in Appendix 3.

I've developed this spreadsheet into the MersAG_Universal Node Capacity Calculator which can help calculate how many of each expansion board are needed and, for a larger and/or more complex layout, how many nodes are needed to control the servos, relay-activated frogs, signals, sensors and other inputs/outputs on a particular layout. This spreadsheet can be completed online at On-line Google sheet address

I calculated that, for my layout, I'd need 4 nodes each with 1 x PCA9685 and 4 x MCP23017. Specifically, for my Colliery region, I'd need :

- 10 PCA controlled servos (for turnouts and animations) of the 16 pins on a PCA9685, and
- 55 MCP input/output pins (for frogs, signals, lighting and detectors) requiring most 64 pins on the 4 MCP32017 expansion boards.

The 8 Native pins on the ESP32 that are available would also give me spare capacity on the node to add additional features that I'm bound to have forgotten or not planned to include!

I also planned to use an 8 channel relay board to switch my 8 frogs and a 6 channel relay board : 4 channels to switch lights and 2 reserved for animations (actually I planned to use 2 x 8 channel relay boards but ordered a 6 channel board by mistake!). Both of these boards are Active Low Operation, meaning they switch "ON" when receiving a LOW (0v) signal. I'll add a 5v supply to power the PCA9685 and both relay boards and a separate 3.3v supply for the MCP boards as taking the 3.3v from the ESP32 would be too demanding. I chose to use a buck converter to step the 5v down to supply this 3.3v.

3.3 Defining Unique Addresses for the Expansion Boards

I'd only need 1 PCA9685 which will have the default address of 0x40. But all 4 MCP23017 boards will be supplied with the default address of 0x20 so 3 of these boards will need to be configured with the unique addresses of 0x21, 0x22 and 0x23.

To configure an MCP23017 with the address 0x21 I'd need to use a soldering iron to melt the solder retaining the resistor across addressable pins A0 (see Fig 3.1), remove the resistor (typically it'll stick to the soldering iron), and solder across from VCC to the centre pin as shown in Fig 3.2, making sure that the solder doesn't touch GND.

This procedure was repeated for pin A1 (for address 0x22) and for both pins A1 and A2 (for address 0x23).

When done, I checked with a volt meter that the VCC pins were connected to their respective centre pins and that they were not shorted to ground.

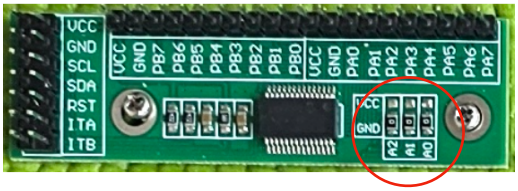


Fig 3.1 MCP23017
showing configurable address pins

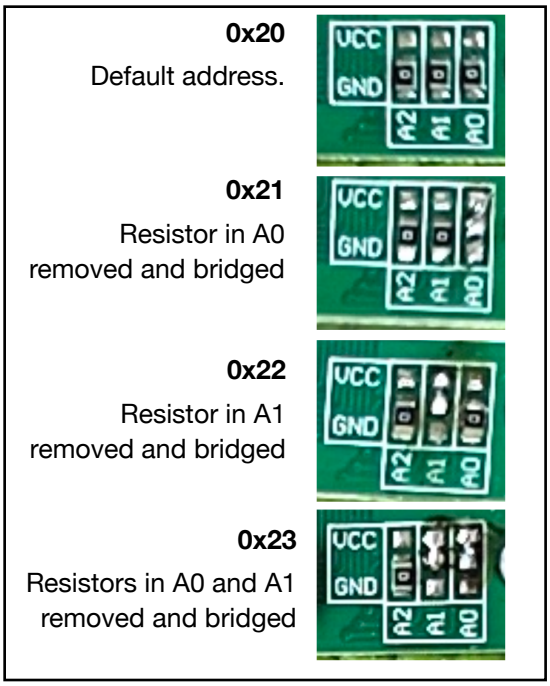


Fig 3.2 Defining the address for each of the 4 MCP23017 boards

3.4 Positioning Components on the Node Board

I started by positioning my components on the node board as shown in Fig 3.3. Of course you can build your own node as you wish and to accommodate the specific components you need.

Fig 3.3 shows the ESP32 on its shield and the expansion boards with their unique addresses. In addition to these “standard” node components I added an 8 Channel Relay Board to control my frogs and a 6 Channel Relay Board to control my animation servos and lights.

I positioned the expansion boards to have their connector pins around the outside of the node board as far as possible to make wiring easier and “cleaner”. By the time this node board is finished there’s going to be a lot of wires!

I’ve left a space at the top left of the board to install power connectors and a transceiver that I’d need to add to each node when I come to adding more nodes : I'll be covering this in Part 4 of this User Guide. Lot’s to get on with in the meantime!

I also added a 5V supply and the CA1235 Buck regulator board which I configured to take the 5V supply and convert it to a 3.3V output supply the MCP23017s.

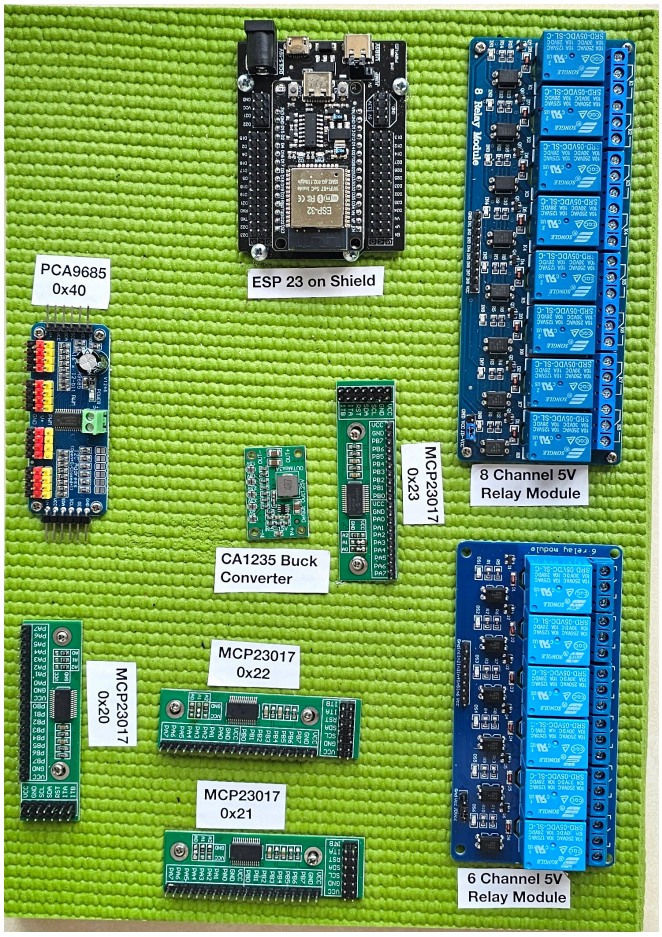


Fig 3.3 My node laid out with components.

3.5 Wiring the I²C Bus

Rather than “hard wiring” (soldering) my bus and supply wires I chose to use 5 and 10cm female Dupont cables to make life easier and to enable reconnecting any wires that have not been connected correctly! Once all connections have been tested I'd hot-glue them to fix them in place.

I started by wiring the I²C bus using green wire for SDA and yellow for SCL (these are commonly used colours for I²C bus wires).

Referring to Fig 3.4 I connected :

1. a green cable from D21 on the ESP32 shield to SDA on the PCA9685
2. a yellow cable from D22 on the ESP32 shield to SCL on the PCA9685
3. these 2 cables from the other set of pins on the PCA9685 to the respective pins on MCP23017 0x20.
4. I continued “daisy chaining” these cables to the other 3 MCP23017 boards.
5. I then connected red and black cables from VCC and GND on the ESP32 shield to their respective pins on the PCA9685
6. and red and black cables from the +’ve and -‘ve connectors from the 3.3V power supply on the Buck converter to VCC and GND, respectively, on MCP23017 0x23.
7. I then “Daisy chained” the power cables from this MCP23017 board to the other 3 MCP23017 boards.
8. and finally connected a black wire from the -‘ve connector on the 5v power supply to GND on the ESP32 shield. This is essential to ensure that the GND on the ESP32 and the 5V supply are common.

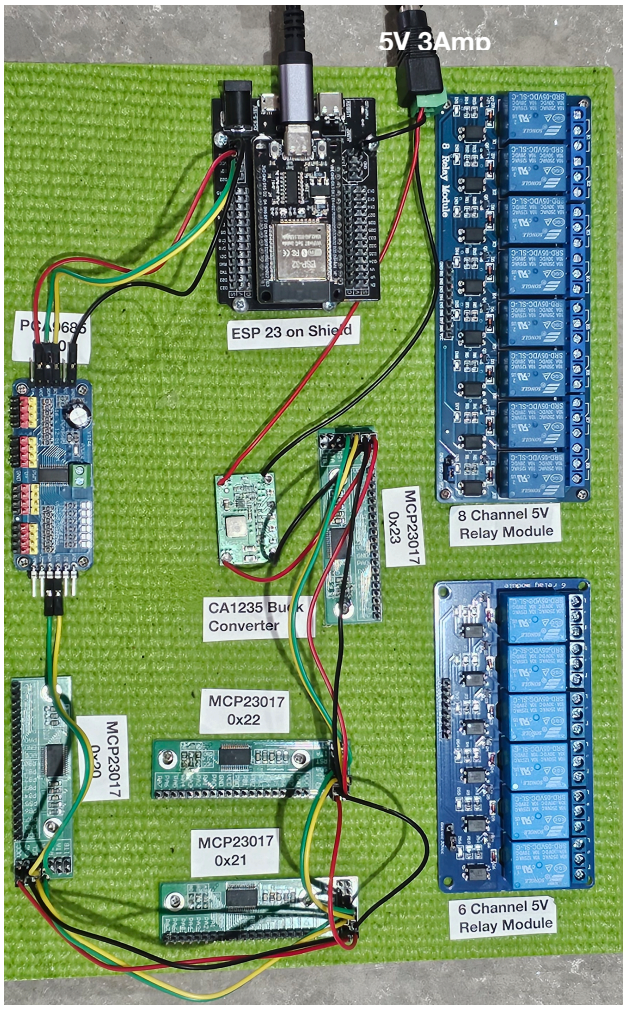


Fig 3.4 I²C bus wired and powered

The I²C bus was now wired. I made a visual check that the wires were not crossed, were on the correct pins and were well connected.

Next I thought that it would be a good time to test my I²C bus and make sure that my addresses were all set correctly. I used an I²C scanner that we’ve included in the “package” when MersAG_Universal is downloaded. So the next job was to download the latest release and test my I²C bus.

3.6 Download the MersAG_Universal Package : including I²C Scanner

If you followed Part 2 you'll already have downloaded MersAG_Universal but you might want to make sure you have the latest package. To download MersAG_Universal (including the I²C scanner sketch) :

1. Click the link : **https://github.com/MersAG-Dev-Org/MersAG_Universal** to open the repository housing the MersAG_Universal sketch.
2. At the bottom right of the opening screen press **Latest Release** (Fig 3.5).
3. At the bottom left of the next screen (Fig 3.6), press **Source Code (zip)** or **Source code (tar.gz)** (depending on your operating system) to download the file.
4. Once the zip file has downloaded, click on it to expand it to a folder and save the folder as required.

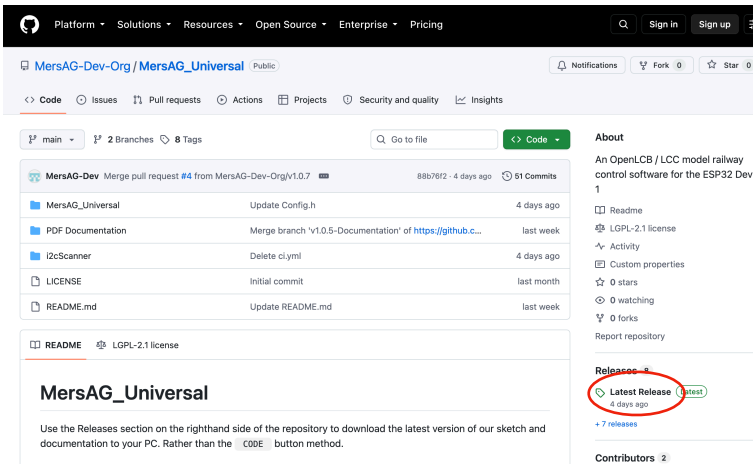


Fig 3.5 Download latest MersAG_Universal

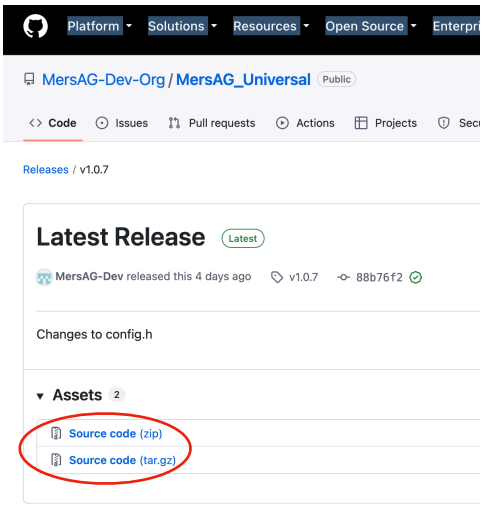


Fig 3.6 Select Source code

Important Note about Screenshots

Screenshots in this guide are taken on an Apple Mac Desktop running MacOS Catalina version 10.15.8. Some layout differences may be seen on other Mac computers and will be seen on Windows and Linux computers.

3.7 Test I²C Bus with i2cScanner

Having downloaded the latest MersAG_Universal release and with the computer connected via a USB cable to the ESP32 (not the Shield) :

1. Open the MersAG_Universal folder and the enclosed i2cScanner folder (Fig 3.7).
2. Double click the i2cScanner.ino sketch to open it in the Arduino IDE.
3. With the :
   - Board set to ESP32 Dev Module,
   - Port correctly set, and
   - Upload Speed set "115200".
4. Upload the sketch to the ESP32
5. In the Serial Monitor you'll see that the sketch is scanning for I²C devices followed by a list of device addresses found. (If you don't see the serial screen you may need to turn your serial monitor on top right of screen)
6. The screen shot shows that my 4 MCP23017s and 1 PCA9685 have been found. If you have a different configuration the screen should, of course, only show the boards you've installed.
7. If no addresses are showing or any are missing you'll need to check all connections to find and fix the cause.

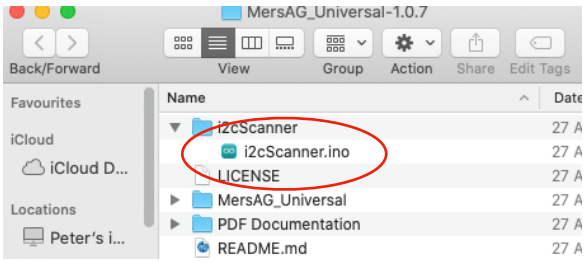


Fig 3.7 Location of i2cScanner.ino



Fig 3.8 Devices found by i2cScanner.ino

Now that I've checked that my I²C bus is connected correctly I can connect my servos to the PCA9685 board(s). At this stage nothing should be connected to the MCP23017 pins: don't worry, you'll understand why later!

3.8 Connecting Servos to the PCA9685 Board

I need 8 servos to control the 8 turnouts on Colliery Region of my layout so I could go ahead and install them directly onto my layout. If your Node Board is readily accessible you can go ahead and do that. But my Node Board will be installed under the layout so I'm going to make a temporary testbed to check the servos, relay boards, signals etc. This will also help when I'm configuring my Node with JMRI as I'll be able to directly see what happens when I make and change settings. It'll also help when I come to configure my other 3 nodes.

You'll see from Fig 3.9 that I've added power distribution blocks and connected 5v and GND lines to my PCA9685. I've also connected my 8 servos to the upper 2 banks of 4 x 3-pin sockets on my PCA9685. A couple of things to note :

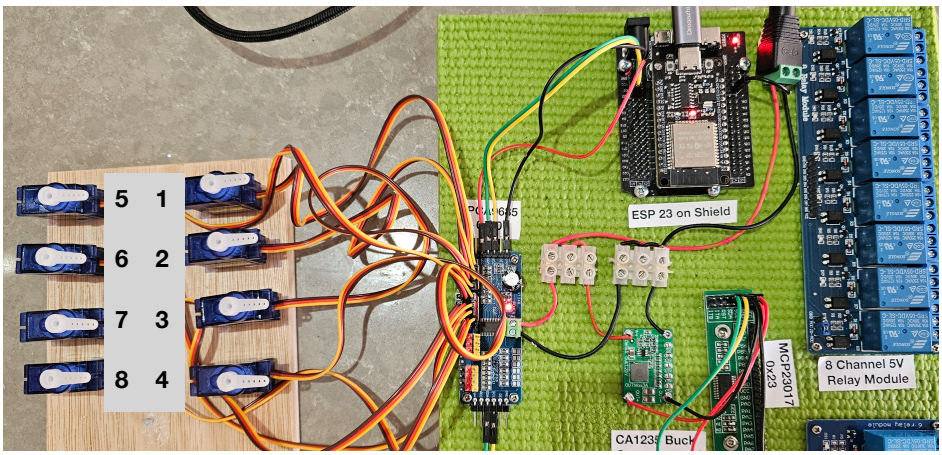


Fig 3.9 Servo Connections

1. The PCA9685 3-pin sockets are not numbered. If you orient your PCA9685 board as shown in the photo, socket 1 is at the top.
2. You must plug the cable in the right way round so the colours match. Sometimes the wire colours may vary but you should be able to see which way round the black/brown (GND), red (VCC) and yellow/white (Signal) cables should be.

So, I now have the I^2C bus working and the servos connected up. I can now configure and upload MersAG_Universal and then get my servos working with the help of JMRI.

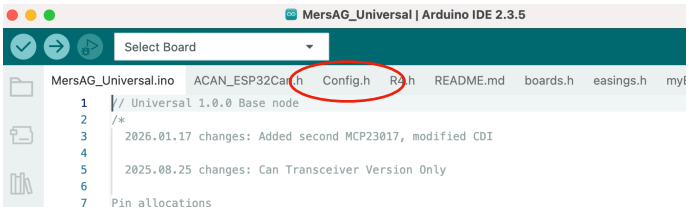
3.9 Configure MersAG_Universal and Install it on the ESP32

Having downloaded the latest release of the MersAG_Universal package in 3.6 I'm now ready to install it on the ESP32. If this is the first time that the sketch has been installed, the following third part libraries **must** be installed using the Arduino Library Manager :

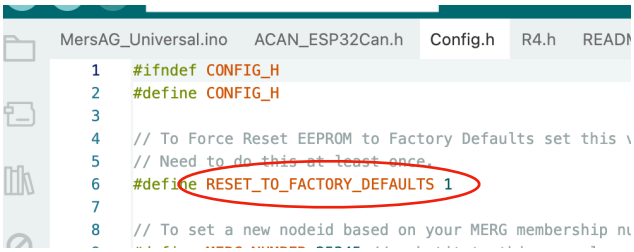
- OpenLCB_Single_Thread by David Harris, Version 0.1.19
- mcp23017 by Bertrand Lemasle tested with version 2.0.0
- ESP32Servo by Kevin Harrigton tested with version 3.2.1
- ServoEasing by Armin Joachimsmeyer tested with version 3.6.0
- ACAN_ESP32 by Mohamed Irfanulla tested on version 3.0.3

Once these Libraries are installed :

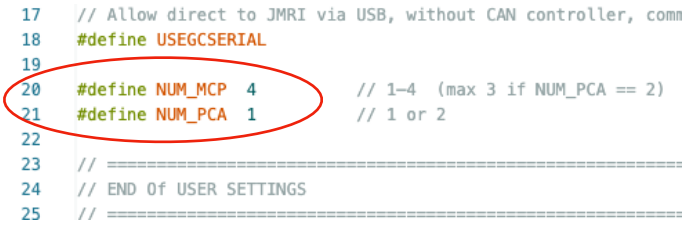
1. Open the MersAG_Universal.ino file in your Arduino ide.
2. Click on the Config.h tab at the top of the sketch.



3. The ESP32's EEPROM **must** be set to factory Defaults. To do this, the end of the line "#define RESET_TO_FACTORY DEFAULTS " must be set to "1". If it's set to "0" you must change it to "1".



4. The default expansion board configuration is 1 x PCA9685 and 4 x MCP23017s, respectively. That's the configuration I'm using. If you're using a different configuration you must define it appropriately. Still in the Config.h section of the sketch, scroll down to the NUM_MCP and NUM_PCA lines and edit the numbers as required.



5. Before compiling and uploading the modified MersAG_Universal sketch, it's worth checking the connection settings. From the Tools menu, make sure that the :
   - Board is set to ESP32 Dev Module,
   - Port is set correctly, and
   - Upload Speed is "115200".

Make changes if necessary and then compile and upload your sketch to your ESP32.

6. After uploading is complete the value of RESET_TO_FACTORY_SETTINGS must be reset **back to O** and **MersAG_Universal uploaded again**.

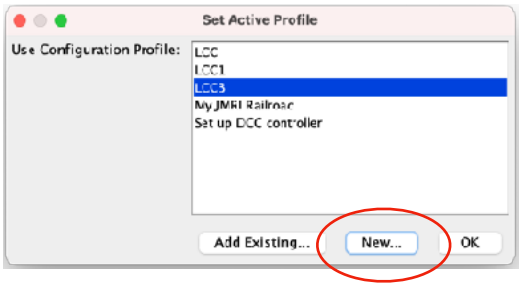
MersAG_Universal is then ready to be configured using JMRI.

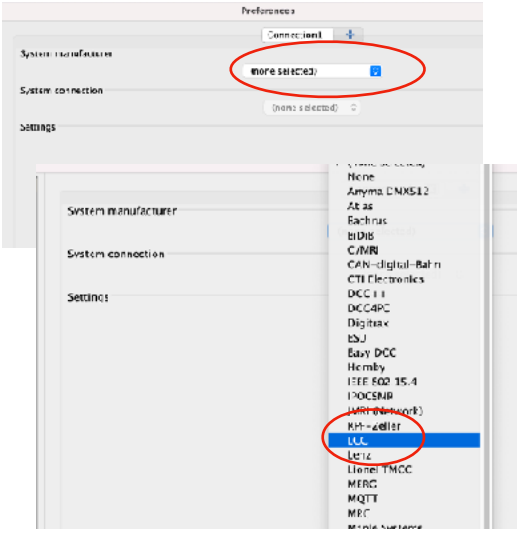
Note : We recommend that you connect your computer to your ESP32 with a USB data cable. I found that I could get away with using a 'standard' USB charging cable on my Dell Laptop (Windows) but not on either my Mac Desktop or laptop.

***Note :** When compiling and uploading a stream of messages in red are displayed. These are not error messages and should be ignored.*

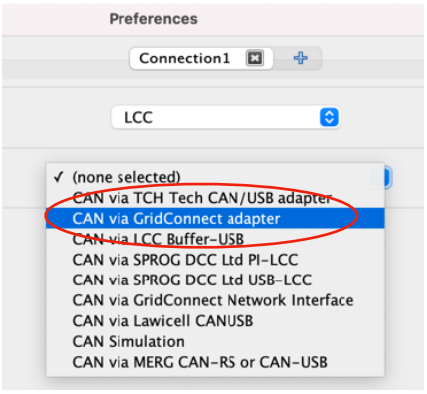
3.10 Configure JMRI to “talk” to the ESP32

If JMRI has already been configured to “talk” to the ESP32 in Part 2, section 2.6, you can skip on to 3.11 below. Otherwise Open PanelPro and, in the Set Active Profile window, select New, enter your Profile name and click OK.

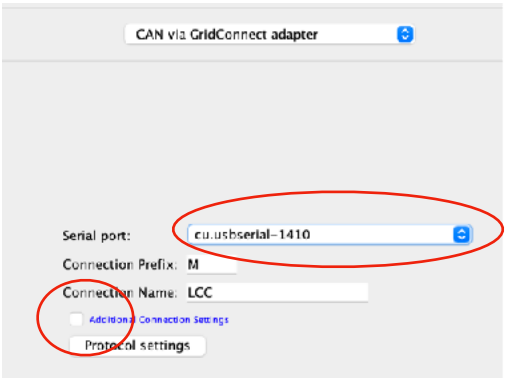




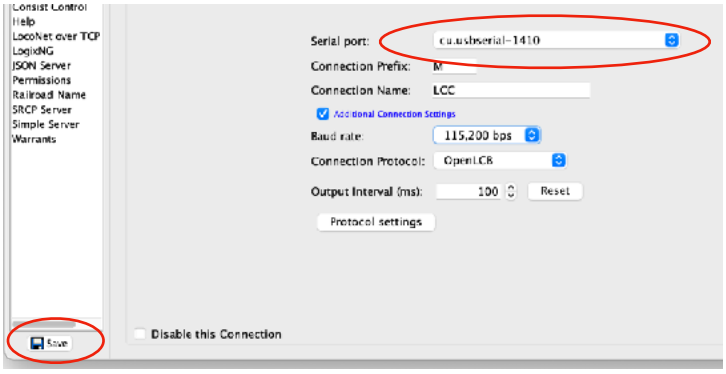
Select the System Manufacturer by opening the drop down window, cursoring down and selecting **LCC**.



Select the System Connection by opening the drop down window, cursoring down and selecting **CAN via GridConect adapter.**



Select your Serial (COM) Port from the dropdown box and then click on **Additional Connection Settings**



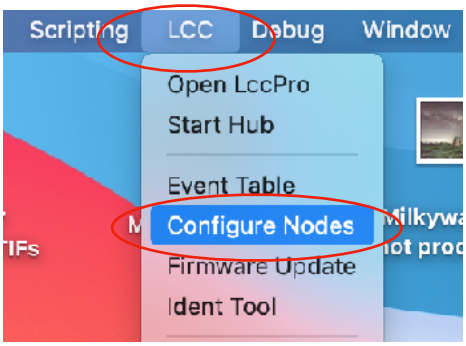
Finally, select **115,200** bps as the Baud Rate and don’t forget to save your configuration by pressing the **Save** button tucked away in the lower left corner.

JMRI has now configured the ESP32 and will automatically restart. It’s then ready for PanelPro to Configure the Node.

3.11 Configure the Node in PanelPro

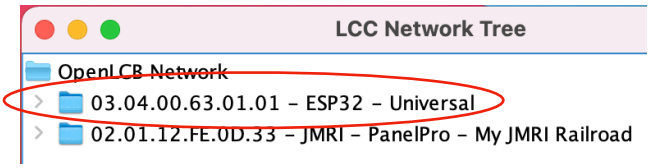
So, I've connected my servos on my test board, I've uploaded the MersAG_Universal sketch to the ESP32 and I've set JMRI up to interact with the ESP32.
Now I want to get PanelPro to configure the PCA(s) on the node to move the servos at a selected speed to and from the "Closed" and "Thrown" positions.

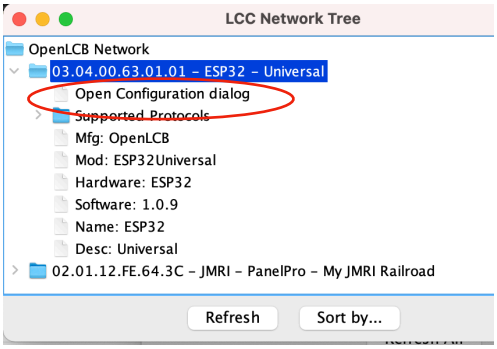
There is now a new tab in the top horizontal menu bar of the PanelPro Window - **LCC**. Click on this tab and select **Configure Nodes** in the drop-down menu.



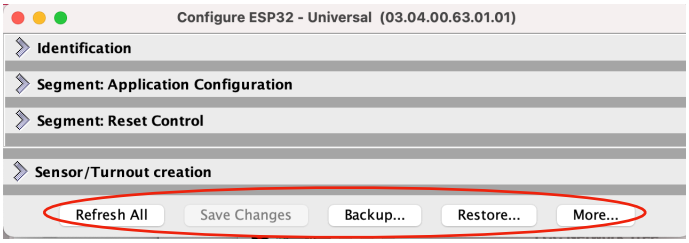
Another window opens up called **LCC Network Tree**.

Expand the **ESP32 - Universal** panel and select **Open Configuration dialog** to open the **Configure ESP32 - Universal** window.





This window allows four options to be expanded and shows a control toolbar at the bottom of the window. This bar features five global action buttons: **Refresh All**, **Save Changes**, **Backup**, **Restore**, and **More**.



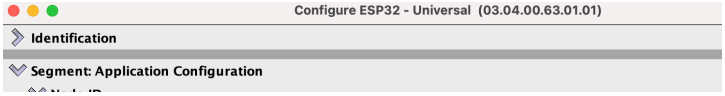
1. Clicking **Refresh All** forces PanelPro to re-query the node over the serial connection, re-reading all parameters stored in the ESP32's EEPROM and updating the displayed values on screen.
2. By default, the **Save Changes** remains greyed out. It becomes active as soon as you alter any field or drop-down menu that requires manual confirmation. You can choose to click **Save Changes** after every edit, or make multiple parameter changes across different segments and save them all at once with this button.
3. The **Backup** Button is essential, especially when teaching peripherals to react to events as we'll see later. When MersAG_Universal is uploaded to an ESP32 for the first time, we recommend that you perform a backup of the blank, default node before making any parameter changes. Clicking **Backup** opens a file dialogue prompting you to choose a file name and storage location on your PC. PanelPro exports the node's entire parameter configuration into a standard `.txt` file. Having a clean baseline backup saves you from having to re-flash the ESP32 firmware from scratch if a configuration gets scrambled during testing.

> **Apple Mac Users.** *I've found that both my Desktop and MacBook Pro restrict free access to save files anywhere on the disk. In addition, files that had been saved and could be "seen" were not being read. I'd created a folders for my MersAG_Universal Backup .txt files and one for my PanelPro saved .xml files which were in a folder called Universal Backups.*
>
> *I moved this folder to the location :* Go - Computer - xxx iMac - Users - yyyyyyyy - Universal Backups where they were both accessible and readable. *(*xxx *and yyyyyyy are labels specifically related to my Mac.)*

4. The **Restore** feature is a life-saver in the event of needing to recover a backup. If an ESP32 board is damaged:
   a. Replace the ESP32 with a new one.
   b. Upload the latest version of MersAG_Universal (see 3.9), ensuring you assign it the original Node ID.
   c. Open PanelPro, navigate back to the **Configure ESP32 - Universal** window and press **Restore**.
   d. Select your most recent `.txt` backup file.

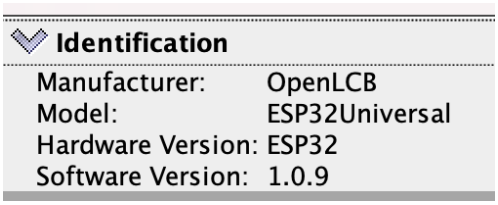
   PanelPro will write your preconfigured servo endpoints, speeds, and I/O event assignments directly into the new ESP32, restoring your layout node to its exact pre-failure operational state within seconds.
5. The **More** button is used for functions that are no longer used or are more easily accessed from other menus and will be explained later.

From the **Configure ESP32 - Universal** window the **Identification** window can be expanded to reveal the core metadata and firmware version details embedded inside the node's software. This information confirms that PanelPro is communicating correctly with the hardware and helps identify which firmware variant is active.



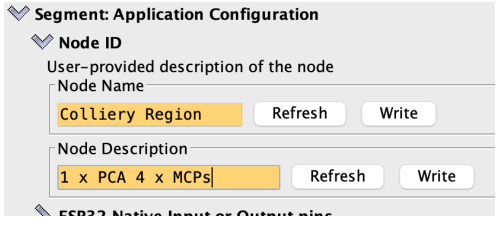
The fields displayed in this segment include:

a. **Manufacturer** : Identifies the base software stack library used to build the node.
b. **Model** : Displays the specific designation for this controller
c. **Hardware Version :** Displays the revision level of the physical board layout and pinout design.
d. **Software Version** : Shows the release version of the compiled software.

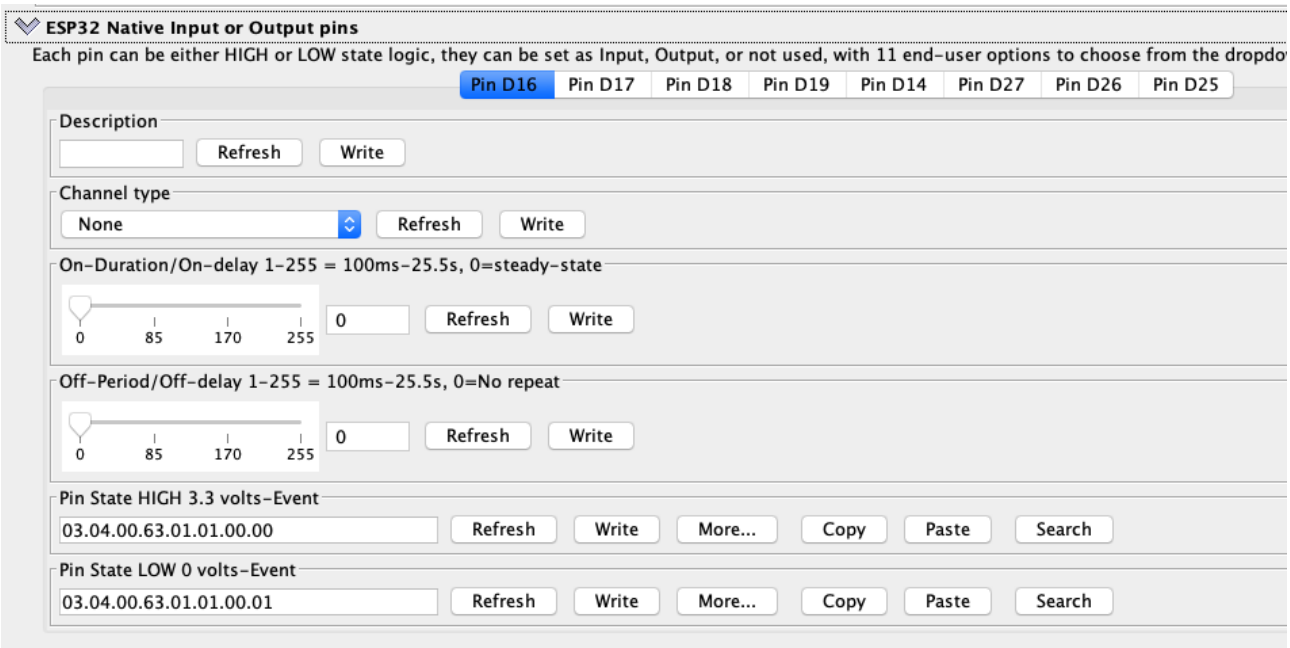


The next window that can be expanded from the Configure ESP32 - Universal window is the **Segment: Application Configuration** window. This window may take a few seconds to open as it needs to gather data that may already have been saved.

The **Node ID** will be open by default and allows a Node Name and Description to be entered. This is particularly useful when you have a network of nodes, as I intend to create in Part 4. I'll name the Node as the region of my layout that this node will control and note how many expansion boards are on it. This will help make sure that I'm addressing the correct node once I've added more.

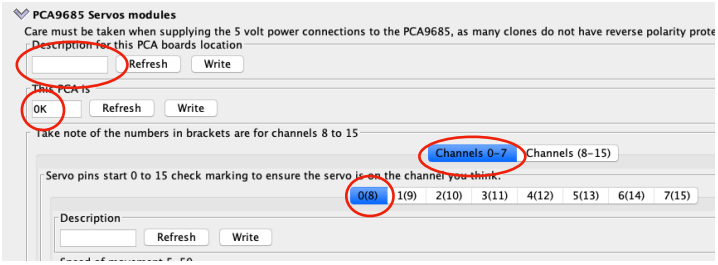


The **ESP32 Native Input or Output pins** window allow the 8 native pins on the ESP32 that are available to users to be configured. These pins are on the ESP32 Shield and designated as pins D16, D17, D18, D19, D14, D27, D26 and D25.
The Native pins are configured in the same way that I'll be configuring my MCP23017 boards later. First, I'll configure my servos and get them working.



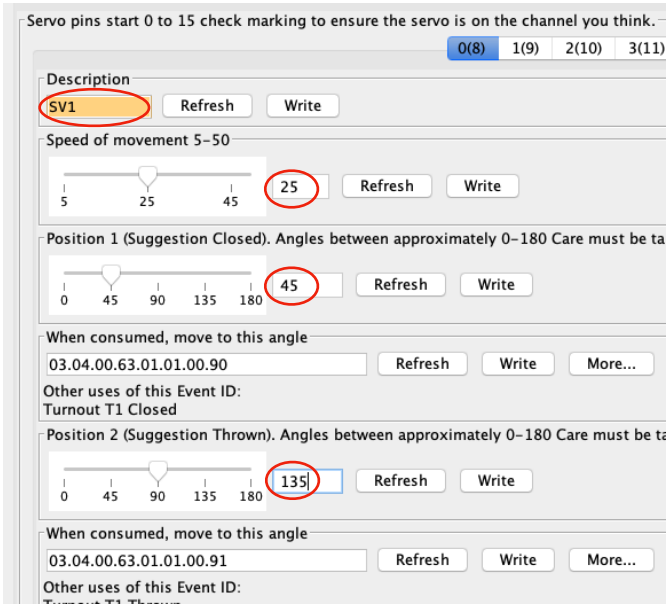
3.12 Configure and Test the Servo Modules in PanelPro

The **PCA9685 Servo modules** window is expanded to configure the servos. If a Node has been configured with 2 x PCA9685 and 1-3 MCP23017 boards, the PCA9685 to be configured must be selected. A Description can be added for each PCA board. Each PCA should show the connected status as being OK.



Channel 0 - 7 and Servo pin 0 are selected by default. Servo pins 0 - 7 correspond to servos 1 - 8 on my test board as they will be on my Node. 8 more servos can be added if required and addressed by selecting Channels (8 - 15) and then Servo pins 8 - 15, respectively.

For each selected servo a description, speed of movement (default = 5), a Position 1 angle and a Position 2 angle (both default to 90) can be entered. Position 1 is normally used to set the angle for when the Turnout is Closed and Position 2 for the angle when the Turnout is Thrown.



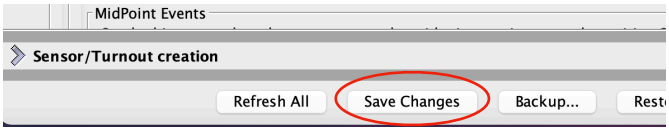
It's worth playing around with the servo parameters to get a good feel of how to edit them and see the responses. First I'll try :

- Description = "SV1",
- Speed = 25,
- Position 1 = 45, and
- Position 2 = 135

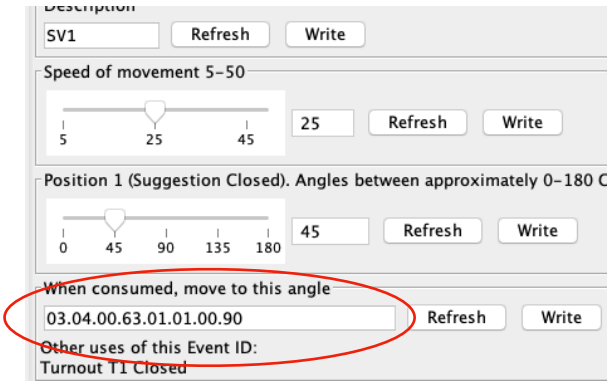
Tip : As you'll see later, it's better to use abbreviated descriptions. e.g. SV for Servo.

If an entry is on a yellow panel or the "Save Changes" button at the bottom of the screen is not greyed-out, either press "Write" to save it or press "Save Changes" to save all changes.

When Write is pressed on either Position the servo will move to that position at the speed entered for that servo.



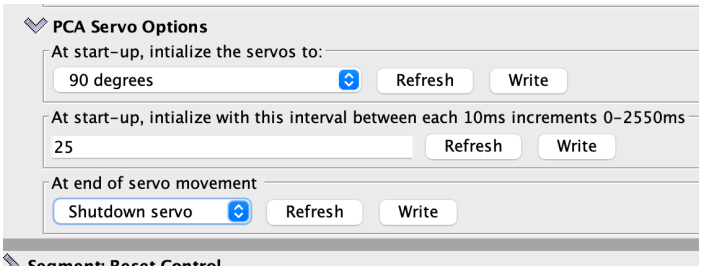
Note : You can change Speed and Position values with either the slider or, more accurately, by entering values and pressing Return.



Also note that, each servo configured will have a **Unique** Event ID for each position. Every individual position tab contains its own unique 64-bit OpenLCB Event ID field (for example, **When consumed, move to this angle 03.04.00.63.01.01.00.90** for Servo 1, Position 1). You'll see later how these events can be used to trigger other events (e.g. switching frogs and/or signals) when they are triggered.

For now, I'll make changes to all of the Servos connected on my Test Board to see how each responds and confirm that all servos are working properly and connected to the correct pins on the PCA9685.

Before I leave my servo configuration, I'll scroll down to the end of the PCA9685 modules window to show the PCA Servo windows options window. For now, I'll select the options as shown :



- 90 degrees
- 25 ms, and
- Shutdown servo.

I'll then Save Changes and make a backup!

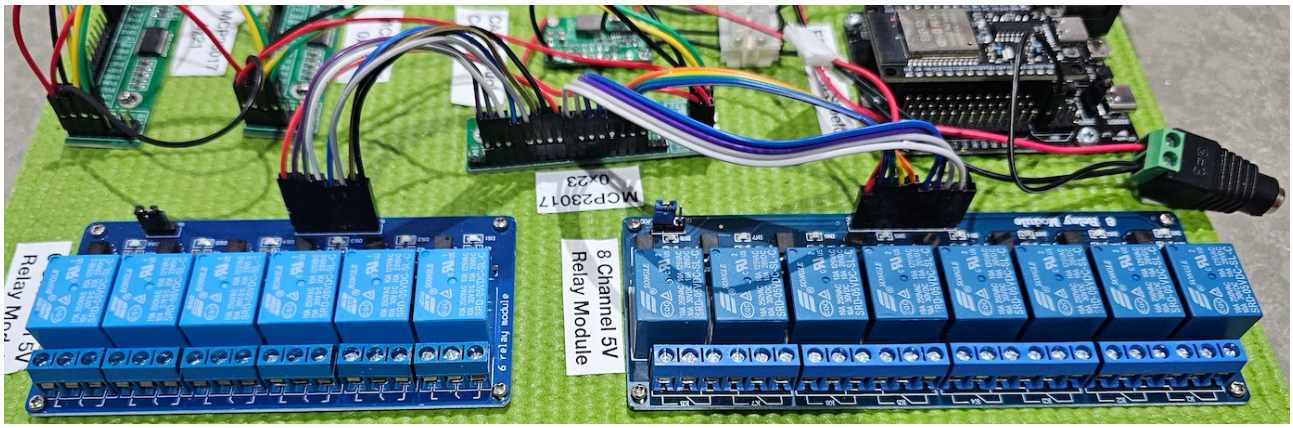
3.13 Planning Pin Connections

Before connecting the MCP23017 pins to the relay boards, LEDs, detectors etc safely, each pin should be set to an Input or Output. This job can be made much easier if you have a spreadsheet or list of all of the peripherals to be connected to all of the pins on the PCA9685 and MCP23017 boards, noting whether they're Inputs or Outputs (PCA9685s can only be Outputs). It'll also prove useful to include a Reference code that will later be assigned to each pin in PanelPro. I've included my Pin Connections Plan in **Appendix 4.**

Any MCP23017 pins that haven't been assigned to an Input or Output should be left at their default state (For all pins PA0 to PA6 and PB0 to PB6 the default is Input with pullup, for all pins PA7 and PB7 their default is Output Phase A.

3.14 Connect the 8 and 6 Channel Relay Boards to the Node Board

For simplicity, I chose to again use Dupont female - female leads to make the connections from pins PB0 - PB7 on the MCP board to pins IN1 - IN8 on the 8 Channel Relay Board. I also connected live and ground cables from the 5V power line to the VCC and GND pins on this board. (the board can be switched with either 5 or 3.3V). Then I connected cables from pins PA0 - PA5 on the MCP board to pins IN1 - IN6 on the 6 Channel Relay Board. Finally I connected VCC and GND pins on the Relay Board to the VCC and GND pins on this Relay Board. (I later reconnected these directly to the 5V power line).



3.15 Configuring the MCP Boards in PanelPro

Open PanelPro and open the MCP23017 16 channel expander window (LCC - Configure Nodes - select LCBNetwork - Open Configuration dialog - expand Segment Application Configuration - expand MCP23017 16 channel expander).

Each of the MCP Boards can be selected and assigned a Description of the board's location.

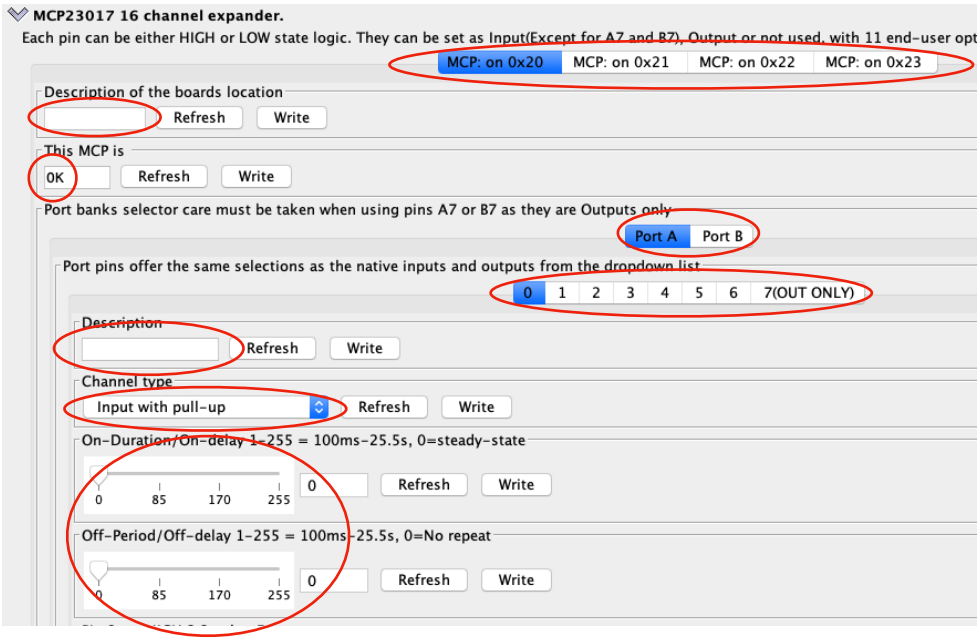
The board with address 0x20 is preselected by default. Check that the selected board shows that "This MCP is" OK. If not then check the other boards. If any are OK it suggests faulty wiring to those boards that are not OK. If all aren't OK check that your 5v power supply to the MCP boards is connected and switched on. Also try quitting PanelPro and restarting.

Once all of the MCP boards are OK the board and the Port to be configured can be selected (Port A for Pins PA0 to PA7 and Port B for pins PB0 to PB7).

A Description, ideally from your Pin Plan, should be entered. As shown in Appendix 3, I've used F1, F2, . . . for Frogs; SG , SG2 its for Signals etc.

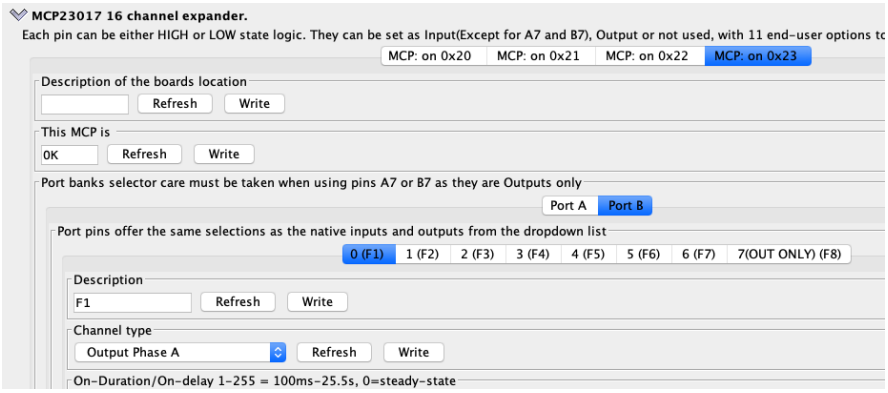
Each pin also needs to be given a Channel Type selected from the Channel type drop-down. The default Channel Type is Input with pull-up. For a description of each of the 7 selectable Channel Types please refer to Appendix 6 : Channel Types.

We'll look at the On-Duration/On-Delay and Off-Period/Off-delay parameters when we're configuring sensors later. These parameters are also described in Appendix 7 : Input / Output Delay Settings.



3.16 Configure and Test Relay Boards to Control Frog Switches

Having configured and tested the servos I'm now ready to configure the MCP23017 with address 0x23 to control the 8 Channel Relay Board to switch my frogs. I connected the MCP pins PB0-7 to pins IN1-8 on my Relay Board in 3.14.

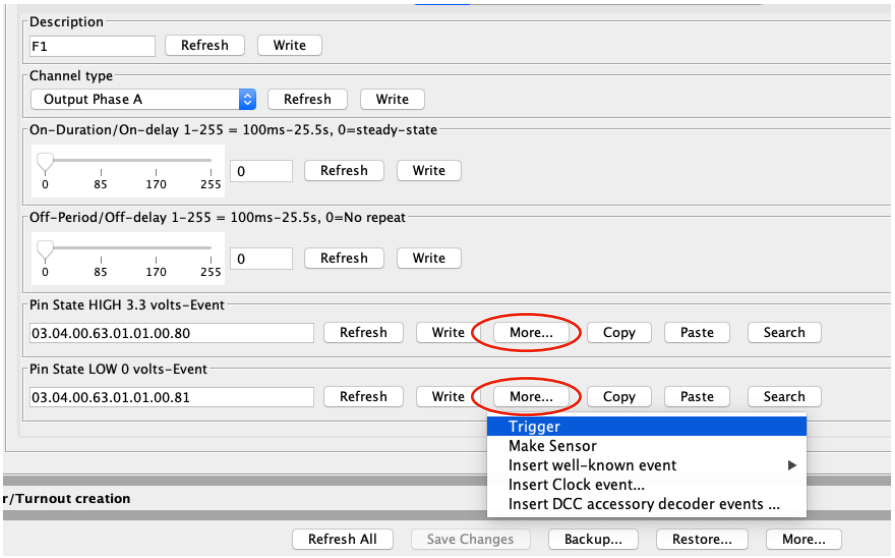


With the 5v supply to the Node board switched ON, expand the MCP23017 window, select MCP 0x23 and check that the board is OK. With Port B and Pin 1 selected I entered **F1** as the Description.

I need to send a negative (LOW) signal to a relay to switch it to ON and a positive signal (HIGH) to switch it to OFF so select **Output Phase A** as the Channel Type.

I didn't need any delays so I left them at their default settings and then saved my changes. I continued entering Descriptions (F2 - F8) and selecting Output Phase A as the Channel Type for the remaining 7 pins.

Having configured all of the MCP 0x23 Port A pins all of the LEDs on the 8 Channel Relay Board were turned on.



With F1 selected, pressing the More Button for the Pin State HIGH 3.3 volts-Event brings up a list of 5 options. Pressing Trigger will turn the indicator LED for Relay 1 OFF. Selecting More - Trigger in the Pin State LOW 0 volts-Event will turn the LED back ON.

If you're using a Relay Board with Active High Operation then triggering the LOW Event will switch the relay OFF and triggering the HIGH Event will switch it ON.

So having configured the Port B pins on MCP 0x23 I configured the Port A pins on the same MCP board and tested the 6 Channel Relay Board. I then connected pins PA0 to PA5 to pins IN1 to IN6 on the 6 Channel Relay Board. I was not planning to use PA6 and PA7 at this time so I left them as their default values (PA6 as Input with Pullup and PA7as Output Phase A) . Once done I checked that the 4 LEDs turned ON and OFF in response to Triggering their respective LOW and HIGH Pin States.

Before moving on to the next step I made another Backup and, for safety, turned the 5v power off.

3.17 Teaching Servos to Control Frog Polarity through the Relay Switches

I was now ready to “Teach” the Relays to switch on and off when the servos controlling them moved the turnouts between their Closed and Thrown positions.

Each time a servo moves :

- to Position 1 or 2, or
- passes through the midpoint, or
- reaches an endpoint

it broadcasts an 8-digit code called a Producer Event. When received by any node on the network, it can cause a response - in this case a relay to switch - by a Consumer. As we'll see later, these events will also be detected by other Consumers on the network to cause other responses such as changing signal lights.

“Teaching” Consumers is achieved by Copying the event code from the Producer and Pasting it to the relevant Consumer(s).

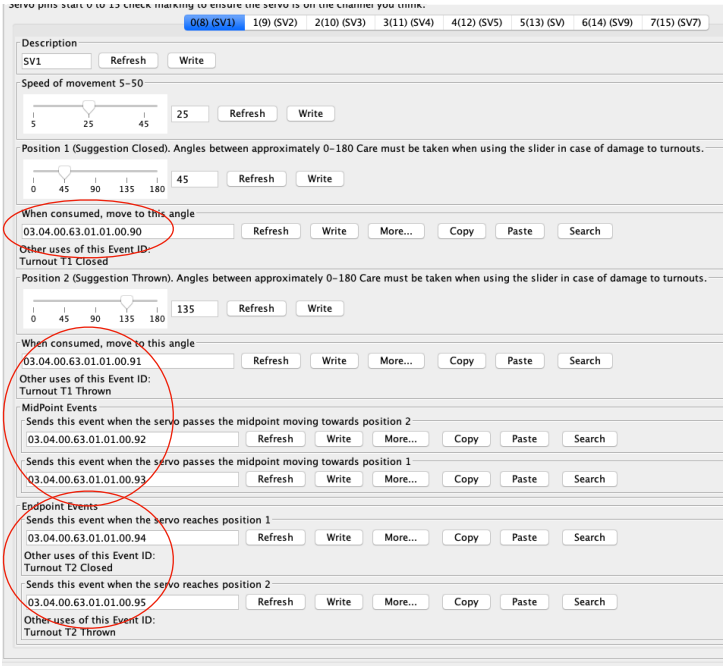
> **WARNING**. Before starting our first “Teaching” session we need to learn about the importance of making Backups Until now, if errors have been made when configuring the various native, MCP and PCA pins or a change of configuration parameters is needed, these parameters can be simply edited or deleted as required.
>
> But that is NOT the case with Events.
>
> Because OpenLCB manages numerous interconnected events it’s not possible to edit or delete entries once they’ve been made. So, if an error has been made when configuring events, it cannot be corrected. Changes to or deletion of Events that have been pasted can’t be made.
>
> The only choice is to either reset the ESP32 EEPROM to factory defaults and start again from an ESP32 that has been reset or to go back to the last “clean” **Backup** and **Restore** the ESP32 EEPROM. Clearly, going back to your the last “clean” backup is, in most cases, the best, easiest option. But you can’t take the few steps back to the last “clean” backup if you’ve not made one!! You’ve been warned!

I’ll start teaching by scrolling down to the PCA9685 Servo modules window and selecting SV1.

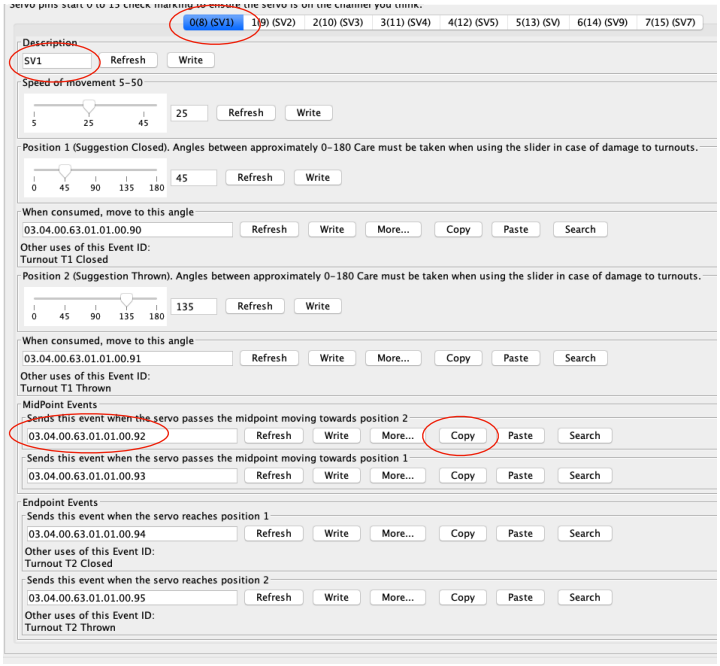
You’ll see 6 event boxes, each of which will broadcast a code at the point that the condition is met by the servo :

1. receiving a signal to **move** to Position 1 (usually Closed), or
2. to Position 2 (usually Thrown);
3. reaching the **midpoint** when moving to Position 2, or
4. to Position 1;
5. reaching the **endpoint** when reaching Position 1, or
6. reaching Position 2.

The safest point to switch frog polarities and avoid shorting will be when the servos/Turnouts are at their midpoints. So I’ll use the Midpoint Events to trigger the Relay to switch the Frog polarity.



To Teach Frog **F1** to switch at the **Midpoint Event** when the servo **SV1** is moving from Position 1 (Closed) to Position 2 (Thrown) I need to :



1. Scroll down to the **PCA9685 Servo Modules** window,
2. Select Pin **SV1**
3. Press the **Copy** button for the **first** of the 2 MidPoint Events.

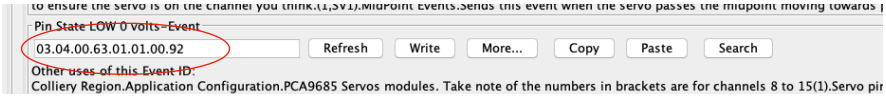
This will copy the event code for when servo 1 is midway from Position 1 (normally Closed) to Position 2 (Normally Thrown) - and the button will display Copied.

At this point I want my Frog (F1) to be switched to positive, so I :

4. Scroll up to **MCP23017 16 Channel Expander Module** window,
5. Select Pin **F1**
6. Scroll towards the bottom of this window.

The relay boards that I'm using are “Low Level Trigger Modules” which means they will switch the Frog voltage to **Positive** when they receive a **LOW** signal (Event) and **Negative** volts when they receive a **HIGH** signal (Event). So, to make the frog switch to **Positive** when the Turnout is midway to the Closed position I need to :

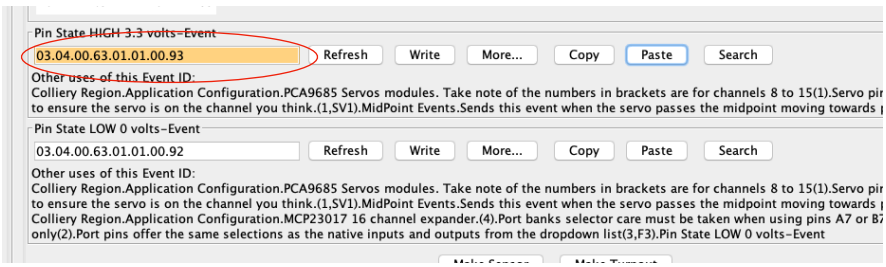
7. Paste the Producer Event that I’d copied in step 3 into the **Pin State LOW 0 volts - Event** window.



When I next move the servo back to Position 1 (Closed) I’ll need to switch the Frog voltage to Negative when the servo/Turnout passes back through the midpoint from the Closed to the Thrown position.

To do this, I need to :

1. go back to the bottom of the **PCA9685 Servo modules** window,
2. Copy the second of the 2 MidPoint Events. This will copy the event code for when servo 1 is midway from Position 2 to Position.
3. Scroll up to MCP23017 window,
4. With **F1**, scroll down and Paste the Producer Event into the **Pin State HIGH 3.3v volts - Event** window. Again, this Event will now be highlighted and so will need to be saved.



To check that with Teaching has worked I’ll go back to the PCA9685 window and move the servo while looking at the LED on the appropriate Relay module. It should turn ON when the servo is midway towards Position 2 and OFF when they’re midway to Position 1. If it’s reversed you may be using an active high relay board.

***Tip** : Notice that the last number in the event code for when servo **SV1** (each servo i.e. Producer will have it's own unique range of codes) is moved to :*

1. *Position 2 is **90** and to Position 2 is **91**,*
2. *At the midpoint moving to Position 2 is **92** and back to Position 1 is **93**, and*
3. *At the Endpoint when the servo reaches Position 1 is **94** and Position 2 is **95***

All these event codes are "paired" with the second in the pay being incremented by 1

So, instead of scrolling back down to copy a "paired" code when it's needed, the first code can be copied and pasted into both "consumed" events. 1 must be then added to the appropriate pasted event.

*Note that event codes are in **hexadecimal** so 99 + 1 = 9A, 9A + 1 = 9B and 9F + 1 = A0 etc.*

This can make copying and pasting much easier.

Once I was happy that F1 had been taught correctly I made a Backup and then proceeded to teach the remaining Relays (F2 to F7), but this time I :

Scrolled to the next Servo (eg SV2)

Copied the Event Code for when the servo reached the midpoint from Position 1 to Position 2,

Scrolled up to MCP23017 0x23

Pasted the Event Code into both Pin State HIGH 3.3-volts Event and Pin State LOW 0-volts Event

Added 1 to the last hexadecimal number in the Event code for the Pin State HIGH 3.3-volts Event

Pressed the Save button, checked the result correct, and made a Backup.

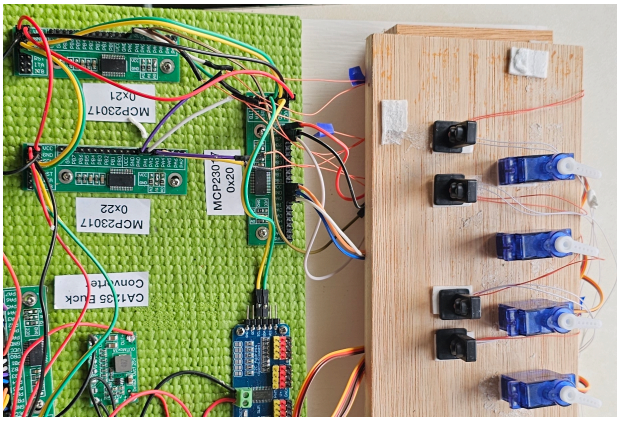
3.18 Configuring the Red and Green Signal LEDs

Next I'm going to work on my Signals. The plan is to install a Red/Green signal on the Closed leg of the Turnout and another on the Thrown side to control locomotives exiting the Colliery Region as shown in Appendix 2.

This will require 16 signals each with a Red and Green LED so I'll need 32 pins :

- 16 on MCP 0x21 to control the Green LED
- 16 on MCP 0x22 to control the Red LEDs.

As a test, I just left 4 servos on my testbed and placed a Red/Green LED signal either side of servos 1 and 3 as shown. I connected the green LEDs to MCP 0x21 pins PA0 and PA1 and PA4 and PA5 and the red LEDs to MCP 0x22 pins PA0 and 1, and PA4 and PA5 PA3.



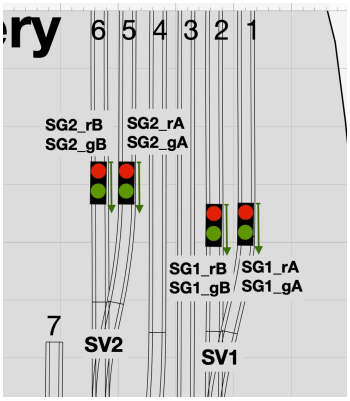
***Note** : It's best to make these connections with the ESP32 disconnected from the computer and with power to the node and testbed turned off to avoid potential shorts.*

Once connections had been made and checked, I connected the ESP32 to the computer, connected and turned on the 5v power supply and ran PanelPro.

I then navigated to the MCP0x21 window and made sure that the board was OK. I entered a description for each pin and set the Channel type to Output Phase A.

You should again describe each pin in the way that works best for you. I've already planned my pin descriptions on my layout plan as shown. I use SG to declare them as a signal, the number is the turnout number, the g or r declare them as red or green LEDs and the A or B declare which of the 2 tracks is selected A is Thrown and B is Closed).

Below is a summary table of descriptions for pins PA0 - PA7. Pins PB0 - 7 will follow the same schema :



Servo and Signal Descriptions

MCP **0x21** pins	PA0	PA1	PA2	PA3	PA4	PA5	PA6	PA7
(all **green** LEDS)	SG1_gA	SG1_gB	SG2_gA	SG2_gB	SG3_gA	SG3_gB	SG4_gA	SG4_gB
MCP **0x22**	PA0	PA1	PA2	PA3	PA4	PA5	PA6	PA7
(all **red** LEDS)	SG1_rA	SG1_gB	SG2_gA	SG2_gB	SG3_gA	SG3_gB	SG4_gA	SG4_gB

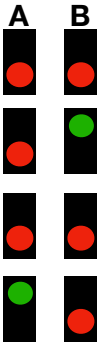
I carefully configured all pins on both MCP0x21 and 0x22. It was a good idea to make a Backup say every 8 pins as I found how easy it was to make mistakes!

I then tested that each LED on the 4 signals on my test board turns :

- ON when I press More ..Trigger for the Pin state HIGH 3.3 volts-Event
- OFF when I press More ..Trigger for the Pin state LOW 0 volts-Event

3.19 Teaching Servos how to Control Signal LEDs

Before I started Teaching the servos how to control the signals I needed to think carefully through what I want to happen and at what point. In 3.17 I taught the frogs to change polarity at the **midpoint** between the Closed and Thrown servo positions. For my signals I thought the following logic would work well (*n* = Signal Number) :

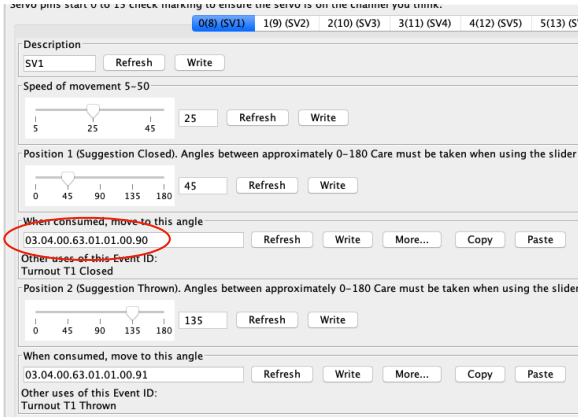


1. When Servo (*n*) is moved to Position 1 (Closed) I'll turn both Red LEDs ON and both Green LEDs OFF.
2. When the servo is at the endpoint, i.e. Closed, I'll turn SG*n*_gB ON and SG*n*_rB OFF
3. When the Servo moves to Position 2 (Thrown) I'll turn SG*n*_gB OFF and SG*n*_rB ON.
4. When the servo is at the endpoint, i.e. Closed, I'll turn SG*n*_gA ON and SG*n*_rA OFF

To do this, for Servo 1 (**SV1**) I need to :

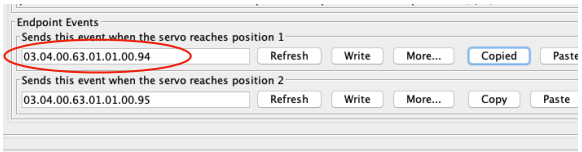
Step 1. Move to Position 1 : Both Red Signals ON and Green OFF

1. In the PCA9685 Servo modules window, select servo Pin 1 (I've described as SV1)
2. Copy the Event Code for "Position 1, When consumed move to this angle".
3. Scroll up to the **MCP Ox21** window, select pin **SG1_gA** and Paste the event code into pin **LOW**.
4. Select pin **SG1_gB** and Paste the event code into pin **LOW**.
5. Scroll up and select MCP **0x22** pin **SG1_rA** and paste the event code into pin **HIGH**.
6. Select pin **SG1_rB** and Paste the event code into pin **HIGH**.
7. Press **Save Changes**.
8. Test (both Red LEDs should be ON and both Green LEDs OFF)and Backup.



Step 2. Endpoint Position 1 Reached : Red A ON, Green B ON

1. Scroll down to the PCA9685 Servo modules window.
2. With **SV1** still selected, copy the Endpoint Event for "when the servo reaches Position 1".
3. Scroll back up and with MCP **0x22** and with **SG1_rB** still selected, paste the event code to **LOW.**
4. Select MCP **0x21** and **SG1_gB** and paste the event code to **HIGH**
5. Test (Red LED on track B should be OFF and Green ON) and Backup.



Step 3. Move to Position 2 : Both Red Signals ON and Green OFF

1. Copy the event for "Position 2, When consumed move to this angle".
2. Paste the event code to **SG1_gB LOW** and **SG1_rB HIGH**.

3. Test (Both Red LEDs ON and both Green OFF) and Backup.

Step 4. Reach Endpoint Position 2 : Red A ON, Green B ON

1. Copy the Endpoint Event for when when servo SV1 reaches Position 2.
2. Paste the event code to **SG1_gA HIGH** and **SG1_rB LOW**.
3. Test ((Red LED on track A should be OFF and Green ON) and Backup.

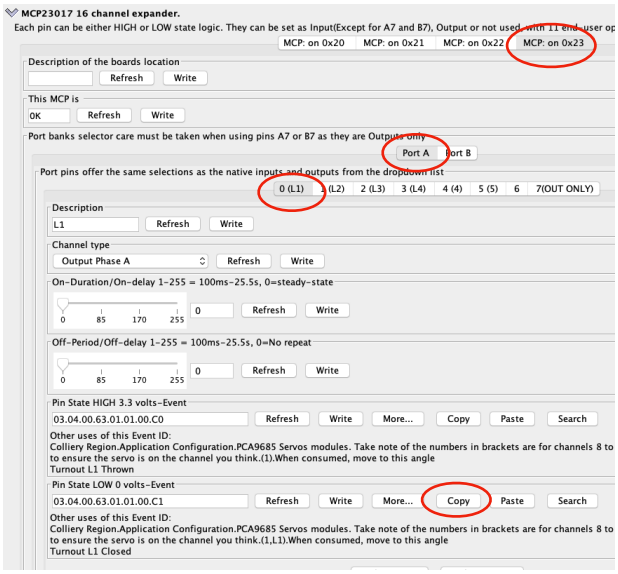
This table summarises the steps detailed above.

Having successfully configured Servo 1 and its associated frog and 2 signals, I'll proceed with configuring and testing the remaining 7 servos on my node before configuring the 6-channel Relay board to switch Lights on and off.

	Signal A			Signal B	
Step	Red	Green		Red	Green
1 Move to Closed	ON	OFF		ON	OFF
2 Reach endpoint Closed				OFF	ON
3 Move to Thrown				ON	OFF
4 Reach endpoint Thrown	OFF	ON			

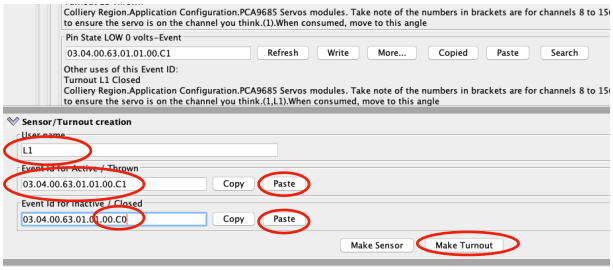
3.20 Configure Relay Board to Switch Station and Other Lighting

The Colliery Region has a Station, Engine Shed, Colliery Works and Access Roads which will all have lighting that will be controlled separately from 4 switches on my Control Panel. I've already configured and connected the 6-Channel Relay Board to be controlled from Pins PA0 - PA3 on MCP23017 0x23. Teaching these pins to switch Relays L1 to L4 will follow exactly the same steps that we followed when teaching the servos to switch the relays to control Frog polarity in 3.17 with the addition of adding button switches on my PanelPro layout.

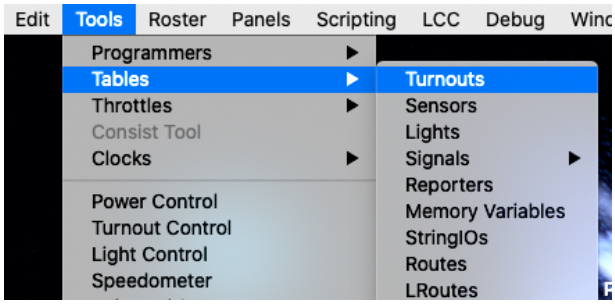


To keep things simple I'll create a turnout in PanelPro and use the Turnout Table to show the Relay Switch turning on and off :

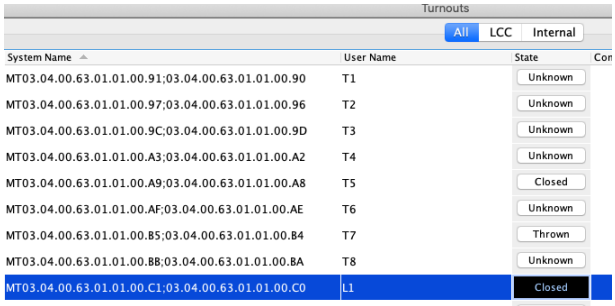
1. Open the MCP23017 16 channel expander window and select MCP 0x23, Port A, Pin L1.
2. Copy the Pin State LOW 0 volts-Event code.



3. Expand the Sensor/Turnout creation table at the bottom of the screen.
4. Enter a User Name and Paste the Event Code into both Event Panels.
5. Edit the Event id for Active/Thrown by adding hexadecimal 1.
6. Press Make Turnout.



7. Select Tools - Turnouts from the PanelPro Menu.



8. Scroll to the retired User Name (L1).
9. Press the State button to toggle the "Turnout" between Closed (ON) and Thrown (ON)

3.21 Configure and Test IR Sensors



LM393

The IR sensors are also reasonably easy to configure and teach. I plan to use LM393s between sleepers and TCRT5000s along platform edges. Both are 3-pin : 5V VCC, GND and data.

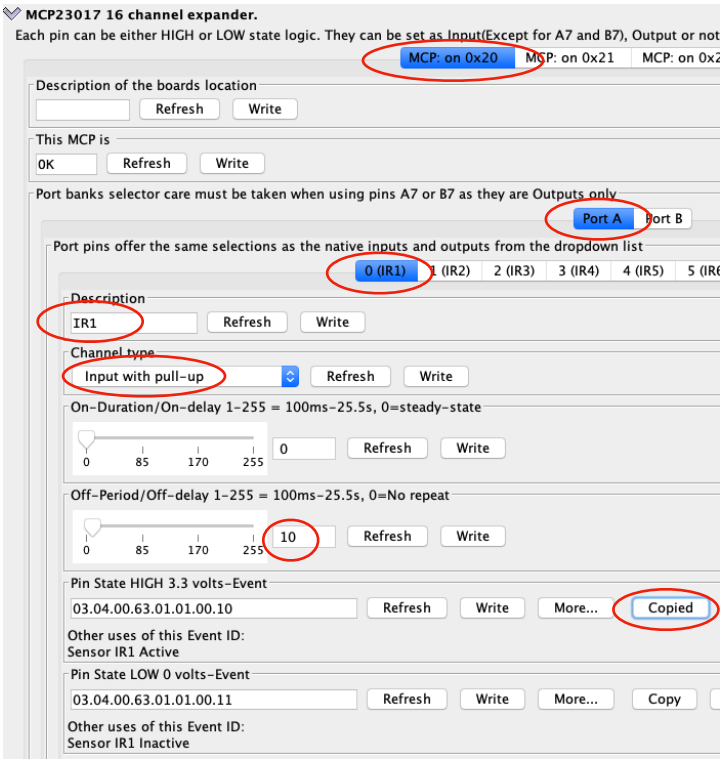
On my plan I need 17 sensors in the Colliery Region to slow locomotives down at various points and stop them at track ends. I plan to use the 16 pins on MCP 0x20 configured as Input with Pull-up but pins PA7 and PB7 can only be outputs. I'll therefore use the 14 pin available on MCP 0x20 and will also use 3 of the Native pins on the ESP32 Shield.

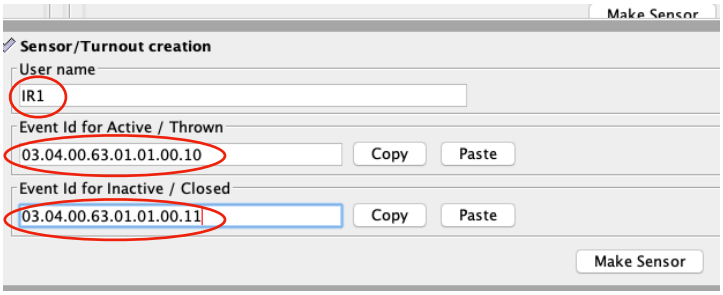


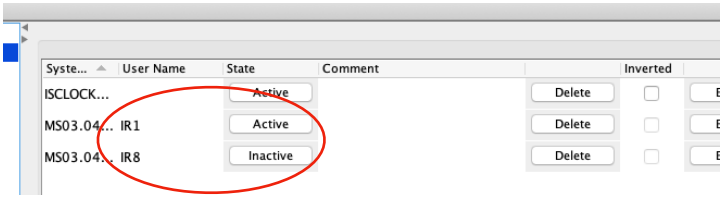
TCRT5000

As a quick test I'll :

1. Connect the data wire from one of each of these sensors to MCP 0x21 Pin PA0 and PB
2. Describe Pins PA0 to PA6 as IR1 to IR7 and Pins PB0 to PB6 as IR8 to IR14.
3. Select Channel Type as Input with pull-up for all 14 pins
4. Enter 10 as the Off-delay for all 14 pins (this provides a 1-second delay to prevent false or rapid re-triggering as cars pass over the sensor).
5. Copy the Pin State HIGH 3.3 volts-Event for any pins you want to test.
6. With the Sensor/Turnout creation window open, Paste the Pin State HIGH 3.3 volts-Event into the Event id for Active / Thrown and this code + 1 into Event id for Inactive / Closed panels.
7. Press Make Sensor
8. In PanelPro, Select Tools - Sensors from the PanelPro Menu
9. Watch the State change from Active to Inactive when you cover the sensor with your hand.

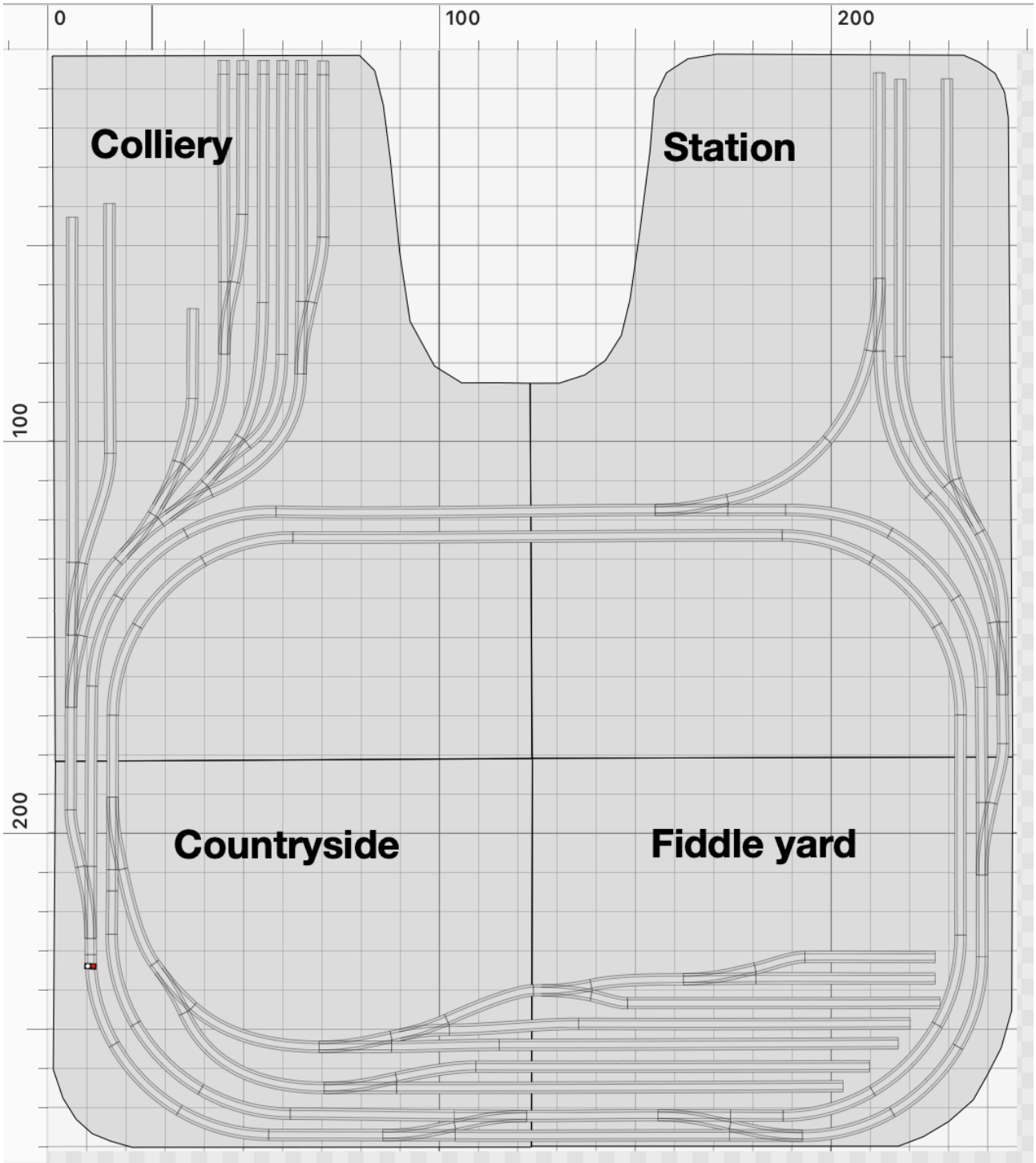




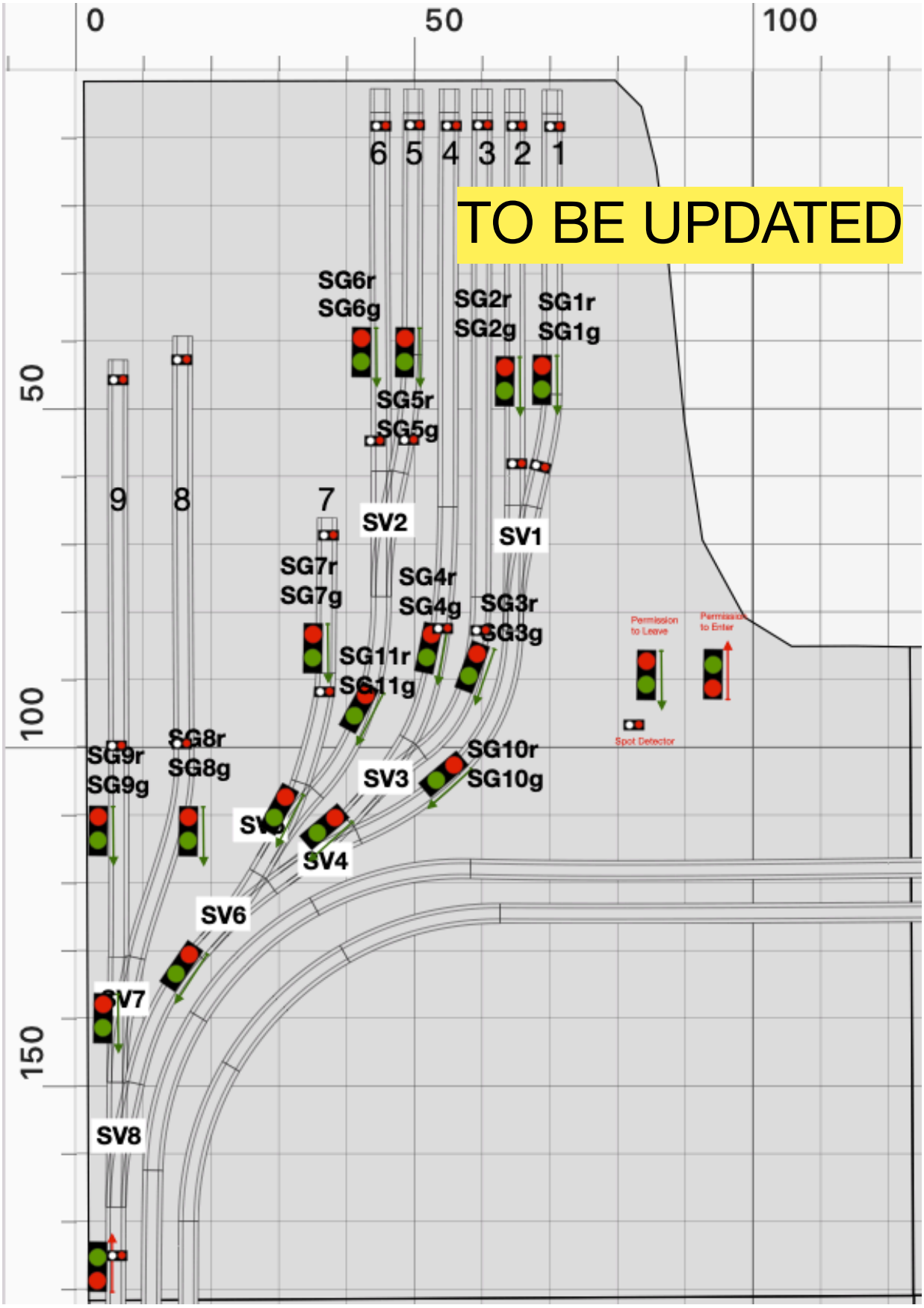


Appendices

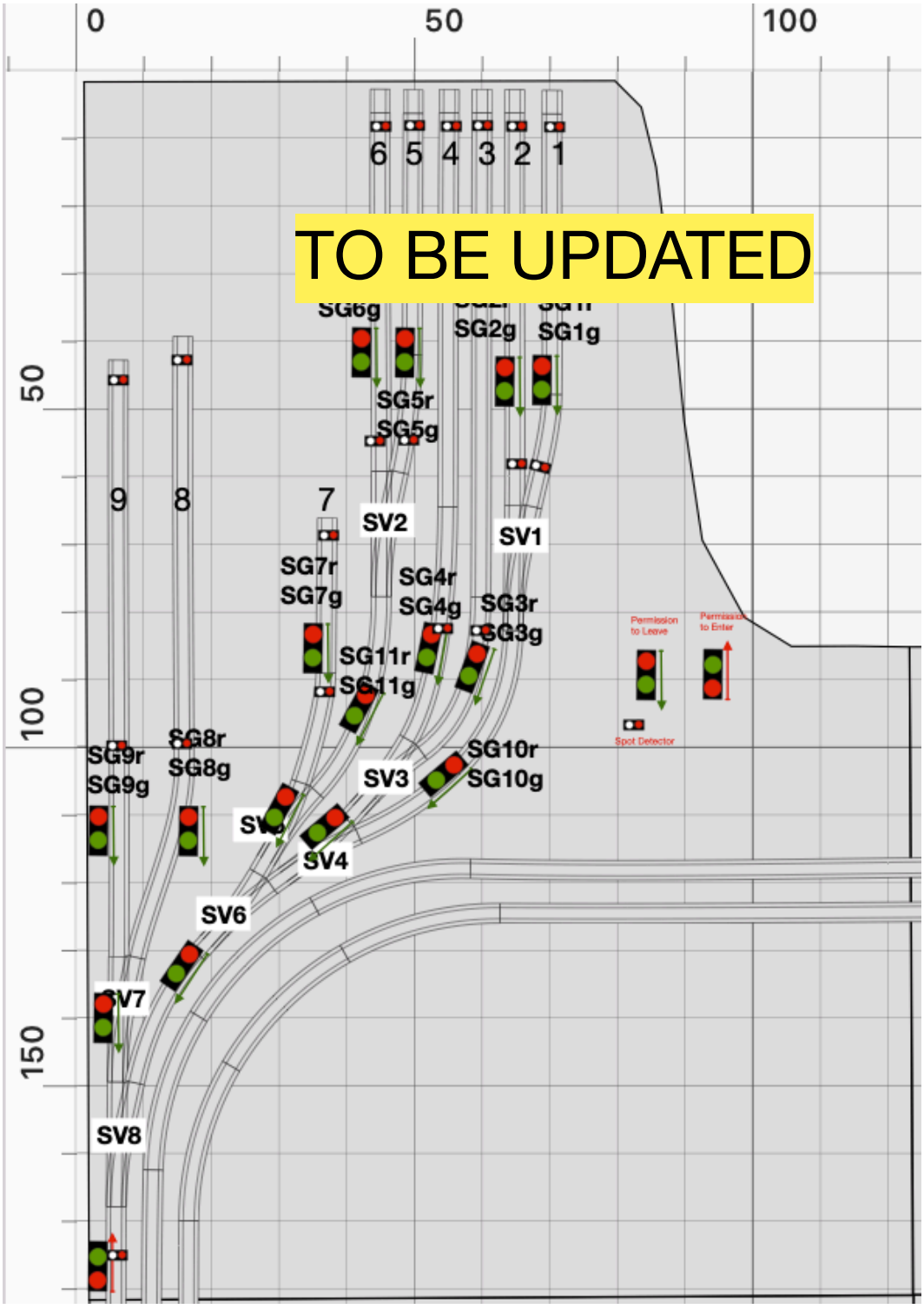
Appendix 1 : Layout Diagram



Appendix 2 : Colliery Detailed Layout Showing Servo and Signal Descriptions



Appendix 3 : Colliery Detailed Layout Showing Sensor Descriptions



Appendix 4 : Expansion Boards needed to Control Layout Regions

MersAG Universal Node Capacity Calculator

Owner :	Pete Colley		Date :	5/10/26						Ver 1.0
Regions	**Note**	**Colliery**	**Fiddle yard**	**Main Station**	**Countryside**					**Total**
PCA Controlled :										
Servo-controlled Turnouts/Points		8	5	4	8	0	0	0	0	**25**
Servo-controlled semaphores		0	0	0	0	0	0	0	0	**0**
Servo-controlled animations		2	0	2	0	0	0	0	0	**4**
Other		0	0	0	0	0	0	0	0	**0**
I/O Pin controlled or read :										
Frog Relay switches	2	8	5	4	8	0	0	0	0	**25**
Semaphore-controlled Signals		0	0	0	0	0	0	0	0	**0**
Red/Green Signals	3	32	12	8	24	0	0	0	0	**76**
Block Detectors		0	8	0	0	0	0	0	0	**8**
Spot detectors		17	10	10	12	0	0	0	0	**49**
Other		0	0	0	0	0	0	0	0	**0**
PCA "pins" needed		**10**	**5**	**6**	**8**	**0**	**0**	**0**	**0**	**29**
PCA expansion boards needed	4	**0.6**	**0.3**	**0.4**	**0.5**	**0.0**	**0.0**	**0.0**	**0.0**	**1.8**
MCP/Native pins needed		**57**	**35**	**22**	**44**	**0**	**0**	**0**	**0**	**158**
MCP expansion boards needed	5	**4**	**2**	**1**	**3**	**0**	**0**	**0**	**0**	**10**
Native Pins Needed	6	**3**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**3**
PCA Boards available		1	1	1	1	0	0	0	0	**4**
MCP Boards available		4	4	4	4	4	4	4	4	**32**
Native Pins available		8	8	8	8	0	0	0	0	**32**
Notes :	1	Node capacity based on 1 PCA and 4 MCP Expansion Boards per node								
	2	Assumes 1 for each Turnout/Point. Edit number if required.								
	3	Generally each Red/Green Signal requires 2 pins and each Red/Amber/Green requires 3 pins per signal.								
	4	If enough PCA pins available for each Region or Total then number is **green**, otherwise **red**.								
	5	If enough MCP boards available then number is **green**, otherwise **orange**. If **total** number of MCP boards available is enough, value will be **green**, otherwise **red**.								
	6	If enough MCP boards available then no native pins are required (but can still be used). If native pins are required and there are enough, number is **green**, otherwise **red**. If **total** number of native pins available is enough, value will be **green**, otherwise **red**.								

Appendix 5 : Pin Connections Plan

Colliery Expansion Boards

		Last Updated :	23/09/2026				
Board	**Connect to**	**Controls**	**Description**	**I/O**	**Channel Type**	**Tested**	**Taught**
PCA0**x40**							
1	Servo 1	Turnout 1	SV1				
2	Servo 2	Turnout 2	SV2				
3	Servo 3	Turnout 3	SV3				
4	Servo 4	Turnout 4	SV4				
5	Servo 5	Turnout 5	SV5				
6	Servo 6	Turnout 6	SV6				
7	Servo 7	Turnout 7	SV7				
8	Servo 8	Turnout 8	SV8				
9	Servo 9	Engine shed door Left	ESD_L				
10	Servo 10	Engine shed door Right	ESD_R				
11	N/A						
12	N/A						
13	N/A						
14	N/A						
15	N/A						
16	N/A						
15	N/A						
16	N/A						
MCP **0x20**	Note that Pins A7 and B7 MUST be OUTPUTS						
PA0	IR Sensor	Sensor 1	IR1	In	Input with Pullup		
PA1	IR Sensor	Sensor 2	IR2	In	Input with Pullup		
PA2	IR Sensor	Sensor 3	IR3	In	Input with Pullup		
PA3	IR Sensor	Sensor 4	IR4	In	Input with Pullup		
PA4	IR Sensor	Sensor 5	IR5	In	Input with Pullup		
PA5	IR Sensor	Sensor 6	IR6	In	Input with Pullup		
PA6	IR Sensor	Sensor 7	IR7	In	Input with Pullup		
PA7	N/A			OUT	Output Phase A		
PB0	IR Sensor	Sensor 8	IR8	In	Input with Pullup		

1

		Last Updated :	23/09/2026			
Board	**Connect to**	**Controls**	**Description**	**I/O**	**Channel Type**	**Teste**
PB1	IR Sensor	Sensor 9	IR9	In	Input with Pullup	
PB2	IR Sensor	Sensor 10	IR10	In	Input with Pullup	
PB3	IR Sensor	Sensor 11	IR11	In	Input with Pullup	
PB4	IR Sensor	Sensor 12	IR12	In	Input with Pullup	
PB5	IR Sensor	Sensor 13	IR13	In	Input with Pullup	
PB6	IR Sensor	Sensor 14	IR14	In	Input with Pullup	
PB7	N/A			OUT	Output Phase A	
MCP **0x21**	Note that Pins A7 and B7 MUST be OUTPUTS					
PA0	Green Signal 1A	Grn LED on T1_A	SG1_gA	Out	Output Phase A	
PA1	Green Signal 1B	Grn LED on T1_B	SG1_gB	Out	Output Phase A	
PA2	Green Signal 2A	Grn LED on T2_A	SG2_gA	Out	Output Phase A	
PA3	Green Signal 2B	Grn LED on T2_B	SG2_gB	Out	Output Phase A	
PA4	Green Signal 3A	Grn LED on T3_A	SG3_gA	Out	Output Phase A	
PA5	Green Signal 3B	Grn LED on T3_B	SG3_gB	Out	Output Phase A	
PA6	Green Signal 4A	Grn LED on T4_A	SG4_gA	Out	Output Phase A	
PA7	Green Signal 4B	Grn LED on T4_B	SG4_gB	OUT	Output Phase A	
PB0	Green Signal 5A	Grn LED on T5_A	SG5_gA	Out	Output Phase A	
PB1	Green Signal 5B	Grn LED on T5_B	SG5_gB	Out	Output Phase A	
PB2	Green Signal 6A	Grn LED on T6_A	SG6_gA	Out	Output Phase A	
PB3	Green Signal 6B	Grn LED on T6_B	SG6_gB	Out	Output Phase A	
PB4	Green Signal 7A	Grn LED on T7_A	SG7_gA	Out	Output Phase A	
PB5	Green Signal 7B	Grn LED on T7_B	SG7_gB	Out	Output Phase A	
PB6	Green Signal 8A	Grn LED on T8_A	SG8_gA	Out	Output Phase A	
PB7	Green Signal 8B	Grn LED on T8_B	SG8_gB	OUT	Output Phase A	
MCP 0x22	Note that Pins A7 and B7 MUST be OUTPUTS					
PA0	Red Signal 9A	Red LED on T9_A	SG9_rA	Out	Output Phase A	

2

Continued ……..

Appendix 5 : Pin Connections Plan (Continued)

		Last Updated :	23/09/2026				
Board	**Connect to**	**Controls**	**Description**	**I/O**	**Channel Type**	**Tested**	**Taught**
PA1	Red Signal 9B	Red LED on T9_B	SG9_rB	Out	Output Phase A		
PA2	Red Signal 10A	Red LED on T10_A	SG10_rA	Out	Output Phase A		
PA3	Red Signal 10B	Red LED on T10_B	SG10_rB	Out	Output Phase A		
PA4	Red Signal 11A	Red LED on T11_A	SG11_rA	Out	Output Phase A		
PA5	Red Signal 11B	Red LED on T11_B	SG11_rB	Out	Output Phase A		
PA6	Red Signal 12A	Red LED on T12_A	SG12_rA	Out	Output Phase A		
PA7	Red Signal 12B	Red LED on T12_B	SG12_rB	OUT	Output Phase A		
PB0	Red Signal 13A	Red LED on T13_A	SG13_rA	Out	Output Phase A		
PB1	Red Signal 13B	Red LED on T13_B	SG13_rB	Out	Output Phase A		
PB2	Red Signal 14A	Red LED on T14_A	SG14_rA	In	Output Phase A		
PB3	Red Signal 14B	Red LED on T14_B	SG14_rB	In	Output Phase A		
PB4	Red Signal 15A	Red LED on T15_A	SG15_rA	In	Output Phase A		
PB5	Red Signal 15B	Red LED on T15_B	SG15_rB	In	Output Phase A		
PB6	Red Signal 16A	Red LED on T16_A	SG16_rA	In	Output Phase A		
PB7	Red Signal 16B	Red LED on T16_B	SG16_rB	OUT	Output Phase A		
MCP 0x23	Note that Pins A7 and B7 MUST be OUTPUTS						
PA0	6 Ch Relay Board Pin IN1	Street Lights	L1	Out	Output Phase A		
PA1	6 Ch Relay Board Pin IN2	Station Lights	L2	Out	Output Phase A		
PA2	6 Ch Relay Board Pin IN3	Works lights	L3	Out	Output Phase A		
PA3	6 Ch Relay Board Pin IN4	Engine shed lights	L4	Out	Output Phase A		
PA4	6 Ch Relay Board Pin IN5	??????		Out	Input with Pullup		
PA5	6 Ch Relay Board Pin IN6	??????		Out	Input with Pullup		
PA6	N/A			In	Input with Pullup		
PA7	N/A			OUT	Output Phase A		
PB0	8 Channel Relay Board IN1	Frog 1	F1	Out	Output Phase A		
PB1	8 Channel Relay Board IN2	Frog 2	F2	Out	Output Phase A		

3

		Last Updated :	23/09/2026				
Board	**Connect to**	**Controls**	**Description**	**I/O**	**Channel Type**	**Tested**	**Taught**
PB2	8 Channel Relay Board IN3	Frog 3	F3	Out	Output Phase A		
PB3	8 Channel Relay Board IN4	Frog 4	F4	Out	Output Phase A		
PB4	8 Channel Relay Board IN5	Frog 5	F5	Out	Output Phase A		
PB5	8 Channel Relay Board IN6	Frog 6	F6	Out	Output Phase A		
PB6	8 Channel Relay Board IN7	Frog 7	F7	Out	Output Phase A		
PB7	8 Channel Relay Board IN8	Frog 8	F8	OUT	Output Phase A		

Appendix 6 : Channel Types

Input

Configures the pin as a standard digital input without activating the ESP32's weak internal pull-up resistor. This mode is ideal when connecting external active electronics or sensor modules that drive their own output signals high and low (push-pull outputs). Common examples on a model railroad include:

- **Active IR Optical Block Detectors:** Modules (such as TCRT5000 or similar turnout/block sensors) that feature their own onboard comparator circuit and actively output a driven logic HIGH (3.3V) or LOW (0V).
- **Current Loop / Occupancy Detectors with Active Outputs:** Occupancy boards that output a clean, buffered 3.3V logic signal directly to the microcontroller.
- **External Electronics with External Pull-down / Pull-up Resistors:** Circuit setups where you have already placed a specific external resistor value on your physical PCB or breakout board, rendering internal pull-ups unnecessary.

Input Inverted

Configures the pin as a standard input without internal pull-ups, but reverses the logic interpretation in software so that a physical LOW signal triggers the High State event, and a physical HIGH signal triggers the Low State event.

Input with Pull-up

Enables the internal resistor to hold the pin at 3.3V (Logic HIGH) when idle. This is the recommended setting for most layout sensors and switches wired to ground. When a sensor or switch closes to ground, the pin is pulled down to 0V (Logic LOW). Reading these state changes triggers the corresponding **Pin High State Event** or **Pin Low State Event** onto the LCC/CAN bus.

Input with Pull-up Inverted

Enables the internal pull-up resistor (holding the pin at 3.3V), but flips the software logic so that pulling the pin LOW to ground fires the High State event instead of the Low State event.

Toggle

Changes state sequentially each time the signal on the pin transitions. Because this setting does not activate the internal weak pull-up resistor, the external circuit or connected device must actively drive the pin HIGH (3.3V) or LOW (0V).

Toggle with Pull-up

Functions identically to the standard Toggle mode, but enables the internal pull-up resistor (holding the pin at 3.3V when open). This allows a simple push-button or switch connected to ground to alternate the output state between HIGH and LOW on each press without needing external resistors.

Output Phase A

Uses the **Pin High State Event** on the LCC network to drive the GPIO pin HIGH (3.3V). This mode is ideal for illuminating a status LED or driving an external active-HIGH relay module, transistor, or indicator. Sending the matching event over the bus immediately sets the pin to 3.3V.

Output Phase A Inverted

Reverses the output logic of Phase A. Receiving the **Pin High State Event** drives the pin LOW (0V), while receiving the **Pin Low State Event** drives the pin HIGH (3.3V). This is particularly useful for active-LOW relay boards or sinking current through an LED wired to a positive supply rail.

Output Phase B

Configures the pin to operate in alternating phase with Phase A outputs. This is specifically designed for dual-indicator setups like level crossing flashers, signal heads, or alternating status lamps. When paired with a Phase A pin receiving the same control commands, Phase B flips to

the opposite state during pulsing or alternating cycles ensuring both lamps flash back and forth rather than blinking together.

Output Phase B Inverted

Reverses the active logic of Output Phase B. Operating opposite to Phase B, this mode ensures that when Phase B is HIGH, Phase B Inverted is LOW (and vice versa). This is useful when driving complex multi-aspect signals, active-LOW dual-LED crossing signals, or alternating relay banks requiring complementary control lines.

Appendix 7 : Input / Output Delay Settings

The delay parameters operate differently depending on whether a pin is configured as an Input (Types 1–6) or an Output (Types 7–10). In both configurations, the values entered for Duration and Period are raw numerical entries ranging from 0 to 255. Entering a non-zero value d multiplies the input by 100 milliseconds:

- Delay (seconds) = (d × 100) / 1000 = d × 0.1 seconds
- Example: A value of 20 equals 20 × 0.1 = 2 seconds.
- Setting any delay parameter to 0 completely disables that timed behaviour.

Output Delay Modes (Types 7–10)

When a pin is configured as an output, the delay fields control pulse times and continuous blinking cycles:

- Duration (Output On-Duration): Determines how long the pin remains in its active (ON) state after receiving an event trigger.
- Period (Output Off-Period): If Period = 0, the pin deactivates after the duration expires and stays off (producing a single pulse).
- If Period >0, the pin switches off for the designated period duration and then automatically turns back on, creating an endless blinking loop.

Example Output Behaviours

Duration	Period	Output Behaviour
0	0	Steady State: Pin changes state and remains there indefinitely.
20 (2s)	0	Single Pulse: Pin activates for 2 seconds upon receiving an event, then turns OFF.
10 (1s)	30 (3s)	Blinking Cycle: Pin stays ON for 1 second, OFF for 3 seconds, repeating continuously

Input Delay Modes (Types 1–6)

When a pin is configured as an input, the delay settings function as debouncing or state-change delay timers before an OpenLCB Event is broadcast onto the bus:

- Duration (Input On-Delay): The delay duration before producing an event after the pin detects a transition to its active state (LOW/pressed).
- Period (Input Off-Delay): The delay duration before producing an event after the pin returns to its inactive state (HIGH/released).

How the Input Engine Handles Delays.

1. Detection: The internal system (produceFromInputs()) continuously monitors physical pin states for any change.

2. Immediate vs. Delayed Action: If the target delay (Duration or Period) is set to 0, the OpenLCB event is generated instantly.

If the delay value is greater than 0, the system schedules a future target timestamp: next[c] = millis() + (d × 100).

3. Execution: The background process (processProducer()) monitors next[c]. As soon as millis() reaches or exceeds the target timestamp, it broadcasts the OpenLCB event to the network and resets the active timer.

Example Input Behaviours.

Duration (On-Delay)	Period (Off-Delay)	Physical Action	Resulting Event Behaviour	Practical Layout Scenario
0	0	Press switch / Trigger sensor	Instant: High State event fires immediately on contact; Low State event fires immediately on release.	Standard fascia push-buttons or toggle switches where instant feedback is required.
5 (0.5s)	0	Press switch	Debounced Press: Pin must stay active continuously for 0.5 seconds before the event fires. Brief electrical noise or accidental taps are ignored.	Mechanical track switches prone to contact bounce, or noisy opto-sensors.
0	30 (3s)	Train leaves block detector	Delayed Release: Event fires instantly when a train enters the block, but waits 3 seconds after the train leaves before sending the cleared event.	Block occupancy detection to prevent signals flicking back to green between car couplers.
20 (2s)	50 (5s)	Sensor triggers briefly	Full Delay Buffer : Requires a sustained 2-second detection before sending the “occupied” event, and stays “occupied” for 5 seconds	

References

CDI / Configuration Description Information

An XML-based feature built into OpenLCB nodes that tells JMRI how to display configuration settings (sliders, drop-downs, names) without requiring custom PC software.

Event IDproducer

A unique 64-bit number broadcast across the OpenLCB network representing a specific state change (e.g., "Turnout 4 Thrown" or "Block 2 Occupied"). Nodes emit or listen for these messages without needing to know which specific board sent them.

Producer/Consumer Model

The Producer/Consumer (P/C) model in OpenLCB is a powerful system control model that allows multiple nodes to control the layout without needing to know each other's configurations. In this model, a node that produces an event is called a Producer, and a node that consumes the event is called a Consumer. This model is based on the idea that one or more Producers can cause one or more Consumers to perform actions, providing flexibility and knowledge independence among the nodes involved. The Producer/Consumer Event ID included in a PCER message is an 8-byte unique identifier that ensures the uniqueness of the event. This model allows for the addition of more Producers or Consumers without concern for side effects to the other nodes involved in the event.

Producer events are messages sent by nodes to control the layout. These events are produced by nodes that trigger actions such as turning lights on or off, controlling turnouts, or signaling other nodes. The producer/
consumer model allows multiple nodes to produce and consume these events, enabling a flexible and decentralised control system. Each event is identified by a unique Event ID, which is assigned by the user to represent specific concepts or actions. This model is powerful because it allows for the addition of new devices to the layout without altering the configuration of existing nodes, making it suitable for large layouts and modular setups.

Consumer events are actions taken by nodes in response to events they consume. These actions can include turning lights on or off, changing signal states, or controlling other devices on the layout. The Producer/Consumer model allows multiple nodes to act on the same event, providing flexibility and power to the system. For example, a node that consumes "Set Lights to Day" can turn on all lights, while another node that consumes "Set Lights to Night" can turn them off. This model is powerful because it allows for complex control without the need for each node to know the other nodes' states or actions.